République Tunisienne
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

[Nom de l'établissement]

[Département / filière]

RAPPORT DE STAGE D'ÉTÉ

Conception et réalisation d'une plateforme web de gestion du Tennis Club de Sayada (TCSAY)

Membres, groupes d'entraînement, réservations de terrains, cotisations, reçus et salaires

Réalisé par
Ayoub Mhenni

Organisme d'accueil
Tennis Club de Sayada (TCSAY)
Sayada, Monastir

Encadrant professionnel
[Nom, fonction au club]

Encadrant universitaire
[Nom]

Période du stage : [du ... au ...] · Année universitaire [2025-2026]

Remerciements

Je tiens à remercier le bureau du **Tennis Club de Sayada**, et en particulier [Nom de l'encadrant professionnel], pour m'avoir accueilli, pour la confiance accordée et pour le temps consacré à m'expliquer le fonctionnement du club : inscriptions, groupes, encaissements, réservations et salaires.

Je remercie également [Nom de l'encadrant universitaire] pour son suivi et ses conseils, ainsi que les entraîneurs et les parents qui ont accepté de tester les premières versions de l'application.

Enfin, je remercie ma famille et mes proches pour leur soutien tout au long de ce stage.

Résumé

Le Tennis Club de Sayada gère ses membres, ses groupes d'entraînement, ses encaissements et ses salaires sur des cahiers papier, avec des paiements en espèces. Ce stage a consisté à concevoir et réaliser une **plateforme web** qui remplace ces cahiers : gestion des joueurs, parents et entraîneurs, groupes et planning des terrains, réservation de terrains par demi-heure, cotisations par tranches avec réductions, reçus imprimés sur le carnet du club, présences, salaires, notifications et import des listes Excel existantes.

L'application repose sur une API **NestJS** (TypeScript) et une base **PostgreSQL** accédée avec **Prisma**, et sur une interface **React** en trois langues (français, anglais, arabe). Elle distingue quatre espaces (administration, entraîneur, parent, joueur) avec des autorisations réglables par le président, applique quatorze règles de gestion côté serveur, conserve un journal d'audit infalsifiable et protège la base de production.

Mots-clés : application web, gestion de club sportif, NestJS, React, PostgreSQL, Prisma, contrôle d'accès par rôles, sécurité.

Abstract

The Sayada Tennis Club manages its members, training groups, payments and coach salaries in paper notebooks, with cash payments. This internship consisted in designing and building a **web platform** replacing these notebooks: players, parents and coaches, training groups and court planning, half-hour court booking, instalment-based fees with discounts, receipts printed on the club's pre-numbered receipt book, attendance, salaries, notifications and import of existing Excel lists.

The application relies on a **NestJS** API with a **PostgreSQL** database (Prisma ORM) and a trilingual **React** interface (French, English, Arabic). It provides four role-based spaces with permissions set by the club president, enforces fourteen business rules server-side, keeps a tamper-proof audit log and protects the production database.

Keywords: web application, sports club management, NestJS, React, PostgreSQL, Prisma, role-based access control, security.

Sommaire

Introduction générale 8

1. Présentation de l'organisme d'accueil et du projet 9

1.1 Le Tennis Club de Sayada 9

1.2 Étude de l'existant 9

1.3 Problématique et objectifs 9

1.4 Démarche de travail 10

2. Analyse et spécification des besoins 11

2.1 Identification des acteurs 11

2.2 Besoins fonctionnels 12

2.3 Besoins non fonctionnels 13

2.4 Règles de gestion 13

3. Conception 15

3.1 Architecture générale 15

3.2 Modèle de données 15

3.3 Scénarios principaux 17

3.4 Choix de conception 18

4. Réalisation 19

4.1 Environnement de développement 19

4.2 Connexion et espaces 19

4.3 Espace administration 20

4.4 Espace parent et joueur 29

4.5 Espace entraîneur 31

4.6 Langues et téléphone 32

5. Sécurité, tests et déploiement 35

5.1 Sécurité 35

5.2 Protection des données de la base 35

5.3 Tests 35

5.4 Déploiement 36

Conclusion générale et perspectives 37

Difficultés rencontrées 37

Compétences acquises 37

Perspectives 37

Webographie 38

Annexes 39

Annexe A — Extrait : règles du mot de passe 39

Annexe B — Extrait : garde des autorisations 40

Annexe C — Commandes d'exploitation 41

Liste des figures

Figure 1 — Planning des sprints et état d'avancement 10
Figure 2 — Diagramme de cas d'utilisation global 12
Figure 3 — Architecture de la plateforme 15
Figure 4 — Diagramme de classes simplifié 16
Figure 5 — Diagramme de séquence : connexion 17
Figure 6 — Diagramme de séquence : encaissement d'une tranche et émission du reçu 17
Figure 7 — Diagramme de séquence : réservation d'un terrain 18
Figure 8 — Écran de connexion 20
Figure 9 — Tableau de bord 21
Figure 10 — Planning des terrains 21
Figure 11 — Liste des joueurs (affichage en cases) 22
Figure 12 — Liste des joueurs (affichage en liste) 22
Figure 13 — Modification d'un joueur : mode de paiement et réduction 23
Figure 14 — Groupes d'entraînement 24
Figure 15 — Fiche d'un groupe 24
Figure 16 — Paiements des joueurs 25
Figure 17 — Reçus et réglage de l'impression sur le carnet 25
Figure 18 — Réservations (vue administration) 26
Figure 19 — Envoi d'une notification 27
Figure 20 — Import des joueurs depuis Excel 27
Figure 21 — Mises à jour par fichier : groupes et paiements du cahier 28
Figure 22 — Autorisations réglées par le président 28
Figure 23 — Historique des actions (journal d'audit) 29
Figure 24 — Accueil du parent 30
Figure 25 — Paiements par tranches (parent) 30
Figure 26 — Réservation d'un terrain par demi-heure 31
Figure 27 — Notifications reçues 31
Figure 28 — Séances de l'entraîneur 32
Figure 29 — Salaires de l'entraîneur 32
Figure 30 — Paramètres en arabe (de droite à gauche) 33
Figure 31 — Affichage sur téléphone : accueil et paiements 34

Liste des tableaux

Tableau 1 — Abréviations utilisées 7
Tableau 2 — Acteurs du système 11
Tableau 3 — Besoins non fonctionnels 13
Tableau 4 — Règles de gestion R1 à R14 14
Tableau 5 — Environnement technique 19

Tableau 6 — Quelques chiffres du projet 19
Tableau 7 — Mesures de sécurité 35
Tableau 8 — Exemples de scénarios de test 36
Tableau 9 — Commandes d'exploitation 41

Liste des abréviations

Abréviation	Signification
API	Application Programming Interface (interface de programmation)
CIN	Carte d'identité nationale
CORS	Cross-Origin Resource Sharing
CSP	Content Security Policy
DT	Dinar tunisien
HSTS	HTTP Strict Transport Security
HTTPS	HTTP sécurisé (TLS)
IJIN	Compte de la fédération (identifiant joueur)
JSON	JavaScript Object Notation
JWT	JSON Web Token (jeton signé)
ORM	Object-Relational Mapping
PWA	Progressive Web App
RTL	Right-To-Left (écriture de droite à gauche)
SMTP	Simple Mail Transfer Protocol
SPA	Single Page Application
TCSAY	Tennis Club de Sayada
UML	Unified Modeling Language

Tableau 1 — Abréviations utilisées

Introduction générale

La numérisation touche aujourd'hui toutes les structures, y compris les associations sportives. Pourtant, beaucoup de clubs fonctionnent encore avec des cahiers papier : liste des joueurs, groupes, cotisations, reçus, réservations des terrains et salaires des entraîneurs. Cette organisation pose des problèmes d'erreurs, de pertes d'information, de suivi des impayés et de transparence vis-à-vis des parents.

C'est le cas du **Tennis Club de Sayada (TCSAY)**, qui m'a accueilli pour mon stage d'été. Ma mission était de concevoir et de réaliser une plateforme web qui remplace les cahiers du club, utilisable par le bureau (président, agent administratif), par les entraîneurs, par les parents et par les joueurs.

Ce rapport présente ce travail en cinq chapitres. Le premier présente l'organisme d'accueil, le contexte et la démarche. Le deuxième analyse les besoins et les règles de gestion. Le troisième décrit la conception : architecture, modèle de données et scénarios. Le quatrième présente la réalisation à travers les principaux écrans. Le cinquième traite de la sécurité, des tests et du déploiement. Une conclusion dresse le bilan et les perspectives.

1. Présentation de l'organisme d'accueil et du projet

1.1 Le Tennis Club de Sayada

Le Tennis Club de Sayada est une association sportive de la ville de Sayada (gouvernorat de Monastir). Il accueille des enfants dès 4 ans, des adolescents et des adultes, en **loisir** (d'octobre à juin) ou en **compétition** (toute la saison, stage d'été compris). Le club dispose de quatre terrains (trois éclairés et une annexe), d'une équipe d'entraîneurs et d'un bureau bénévole.

Les joueurs sont répartis en **29 catégories** d'âge et de genre définies par la fédération (Lutins, Poussins, Benjamins, Minimes, Cadets, Juniors, Seniors, Vétérans, Entreprise, Padel), puis en groupes d'entraînement avec des créneaux fixes par jour, un terrain et un ou plusieurs entraîneurs.

[Compléter : date de création du club, nombre d'adhérents, organigramme du bureau, palmarès.]

1.2 Étude de l'existant

Avant le projet, toute la gestion se faisait sur papier :

- un **cahier d'inscriptions** par saison (nom, date de naissance, téléphone du parent, groupe) ;
- un **cahier des paiements** (montant, somme versée, reste, n° du reçu) et un **carnet de reçus pré-imprimés** rempli à la main ;
- des **listes de convocation** par groupe, retapées à chaque changement ;
- des réservations de terrains notées à la main, sans vision des créneaux libres ;
- des salaires et des absences d'entraîneurs calculés sur papier.

Cette organisation entraîne plusieurs limites : fautes d'orthographe dans les noms, doublons, difficulté à retrouver qui a payé quoi, absence de relance des impayés, risque de perte des cahiers, aucune information pour les parents, et beaucoup de temps passé par le bureau.

1.3 Problématique et objectifs

La problématique est la suivante : *comment remplacer les cahiers du club par un outil numérique fiable, simple à utiliser par des bénévoles, accessible aux parents, et qui respecte les règles de gestion du club et la protection des données de mineurs ?*

Les objectifs fixés avec le bureau sont :

- centraliser les joueurs, parents, entraîneurs, groupes et saisons ;
- suivre les cotisations par tranches, les réductions, les encaissements et les reçus ;
- permettre la réservation des terrains en ligne, sans double réservation ;
- donner aux parents et aux joueurs l'accès à leurs séances, absences et paiements ;

- permettre aux entraîneurs de pointer les présences et de consulter leurs salaires ;
- garder une trace de chaque modification sensible (audit) et protéger les données.

1.4 Démarche de travail

Le projet a suivi une démarche **itérative** inspirée de Scrum : des sprints courts, chacun livrant une partie utilisable, présentée à un ou deux membres du bureau, puis ajustée selon leurs retours. Un **prototype cliquable** (maquette HTML) a d'abord été réalisé et validé ; il a servi de référence pour le design et les textes de l'interface.

Le code est versionné avec **Git** et hébergé sur **GitHub** (47 commits au moment de la rédaction), avec des messages normalisés (*Conventional Commits*). Un assistant de programmation par IA a été utilisé comme aide au développement (génération de code, revue, tests) ; chaque fonctionnalité a été spécifiée, relue, testée et validée par l'auteur.

> **Remarque :** adapter le paragraphe sur l'assistant IA aux règles de l'établissement ; certaines écoles demandent de déclarer les outils utilisés.

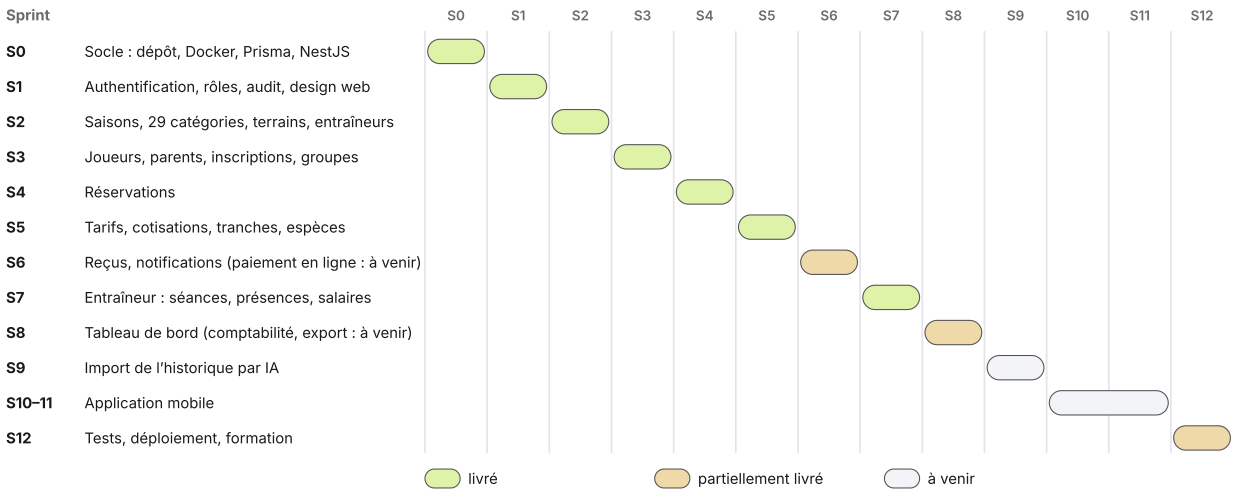


Figure 1 — Planning des sprints et état d'avancement

2. Analyse et spécification des besoins

2.1 Identification des acteurs

Acteur	Rôle et droits principaux
Joueur	Consulte ses séances, absences et paiements ; réserve un terrain en loisir ; reçoit les notifications.
Parent	Comme le joueur, pour **plusieurs enfants** (sélection de l'enfant en haut de chaque écran).
Entraîneur	Ses séances, le pointage des présences de la séance en cours, ses absences et ses salaires ; réserve pour des séances privées. Ne voit jamais les cotisations.
Président	Tous les droits ; règle les autorisations des autres rôles du bureau ; il reste toujours au moins un président actif.
Agent administratif	Joueurs, parents, encaissements, reçus, salaires, réservations, notifications (droits par défaut, modifiables).
Superviseur	Tableau de bord et statistiques (par défaut).
Directeur technique	Groupes, planning, absences et présences des entraîneurs (par défaut).

Tableau 2 — Acteurs du système

Un même compte peut cumuler plusieurs rôles (par exemple directeur technique et entraîneur, ou entraîneur et parent). L'utilisateur choisit alors son **espace** de travail ; les menus et les droits s'adaptent.

2.2 Besoins fonctionnels

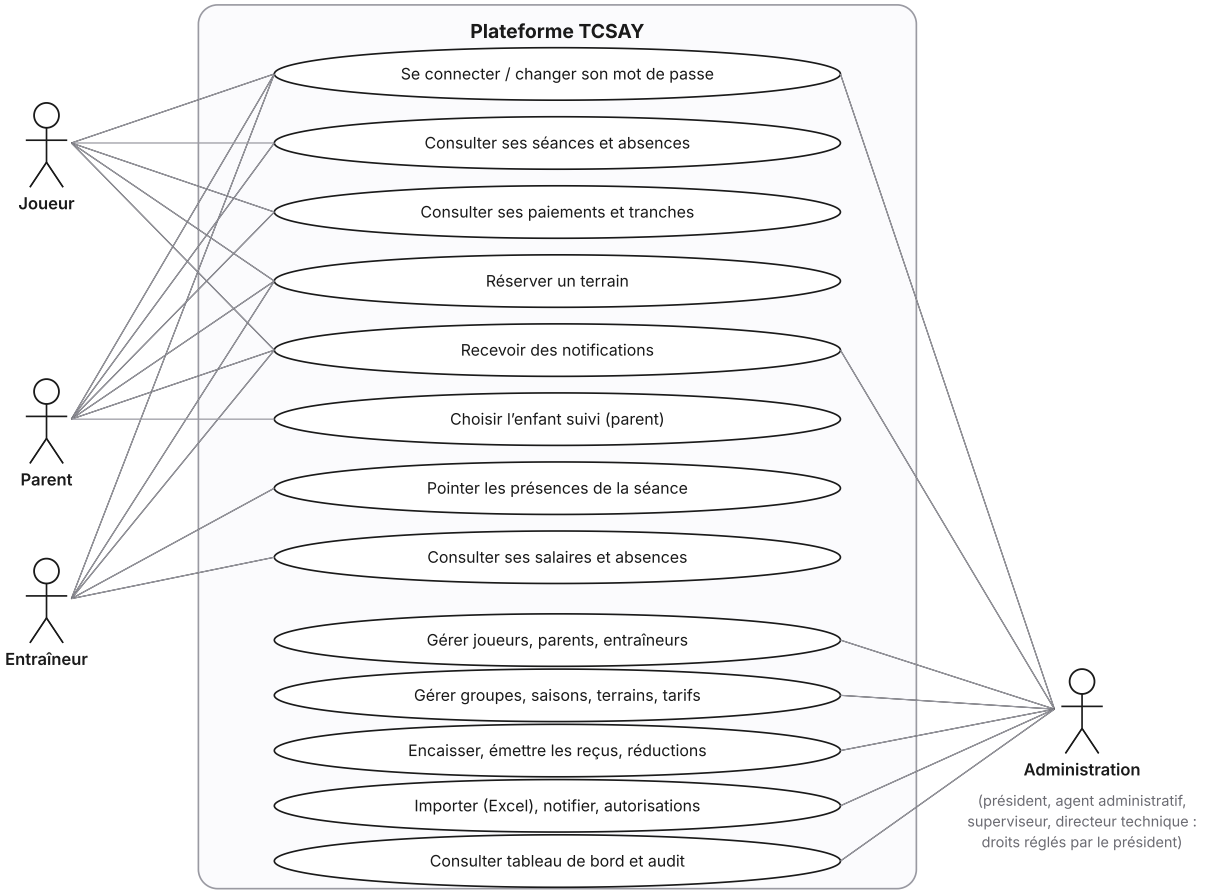


Figure 2 — Diagramme de cas d'utilisation global

Administration

- Gérer les joueurs (création avec catégorie proposée selon l'âge et le genre, code club « 26TCSAY001 », archivage), les parents (liens plusieurs-à-plusieurs), les entraîneurs et le personnel ;
- gérer les saisons (brouillon, active, clôturée, historique), les 29 catégories, les terrains, les groupes et leurs créneaux ;
- définir les tarifs d'entraînement et de terrains ; créer les cotisations, choisir le mode de paiement (comptant, semestre, mois), appliquer des réductions (fratrie, spéciale, exonération) ;
- encaisser (espèces, chèque) et émettre le reçu sur le carnet pré-numéroté ;
- gérer les réservations, les salaires et les absences des entraîneurs (remplacement, séance physique, annulation) ;
- importer les listes Excel existantes (joueurs, groupes, paiements du cahier) ;
- envoyer des notifications ; consulter le tableau de bord, les emails envoyés et le journal d'audit.

Entraîneur, parent et joueur

- Entraîneur : séances à venir et passées, pointage (présent, retard, absent), absences, salaires, réservation de séances privées ;
- parent et joueur : accueil avec la prochaine séance, paiements par tranches, séances, absences, réservation de terrain, notifications, paramètres (mot de passe, langue, thème, compte fédération IJIN).

2.3 Besoins non fonctionnels

Besoin	Réponse apportée
Sécurité	Mots de passe chiffrés, sessions signées, droits vérifiés par le serveur, limite d'essais de connexion, en-têtes HTTP de sécurité, journal d'audit.
Simplicité	Interface claire et aérée, cartes et pastilles de couleur, confirmations avant les actions sensibles, messages d'erreur explicites.
Multilinguisme	Français, anglais et arabe (écriture de droite à gauche).
Accessibilité mobile	Mise en page adaptée au téléphone (une colonne, menu en tiroir).
Fiabilité des données	Règles de gestion côté serveur, contrôle de version contre l'écrasement, sauvegardes, base de production verrouillée.
Protection des données	Données de mineurs : accès limité par rôle, aucune donnée réelle dans les démonstrations, sauvegardes hors du dépôt de code.
Maintenabilité	TypeScript de bout en bout, code commenté en français, migrations de base versionnées.

Tableau 3 — Besoins non fonctionnels

2.4 Règles de gestion

Les règles suivantes ont été définies avec le bureau. Elles sont **toujours vérifiées par le serveur** (jamais seulement par l'interface) ; en cas de refus, l'utilisateur voit un message du type « R8 · … » expliquant quoi faire.

Règle	Énoncé
R1	Une saison clôturée ou historique est verrouillée ; réouverture par l'admin avec motif.
R2	Une seule saison active.
R3	Catégorie modifiable par l'admin seulement ; hors norme : dérogation avec motif.
R4	Ajout dans un groupe : capacité respectée, catégorie correspondante (dérogation avec motif).
R5	Pas de conflit de terrain, de créneau ou d'entraîneur entre groupes.
R6	Salaires d'une saison clôturée : motif obligatoire, anciennes valeurs conservées.
R7	Un paiement encaissé n'est jamais modifié ni supprimé ; le montant dû ne change qu'avant le premier paiement ou avec motif.
R8	Suppression = archivage si l'élément a des liens.
R9	Nom, naissance, genre, catégorie : admin seulement ; un mineur a toujours au moins un parent lié.
R10	Annulation de réservation avec délai (24 h) ; pas de modification d'un créneau passé.
R11	Un terrain avec des réservations futures ne peut pas être désactivé.
R12	Seul l'admin attribue les rôles ; toujours au moins un président actif.
R13	Contrôle de version contre l'écrasement entre deux administrateurs.
R14	Les données importées gardent leur origine et n'écrasent jamais une saisie manuelle sans validation.

Tableau 4 — Règles de gestion R1 à R14

3. Conception

3.1 Architecture générale

L'application suit une architecture **client-serveur en trois couches** : une application web monopage (SPA) écrite en React, une API REST écrite avec NestJS, et une base relationnelle PostgreSQL. Les deux parties sont dans un même dépôt (monorepo) avec les dossiers `api/` et `web/`.

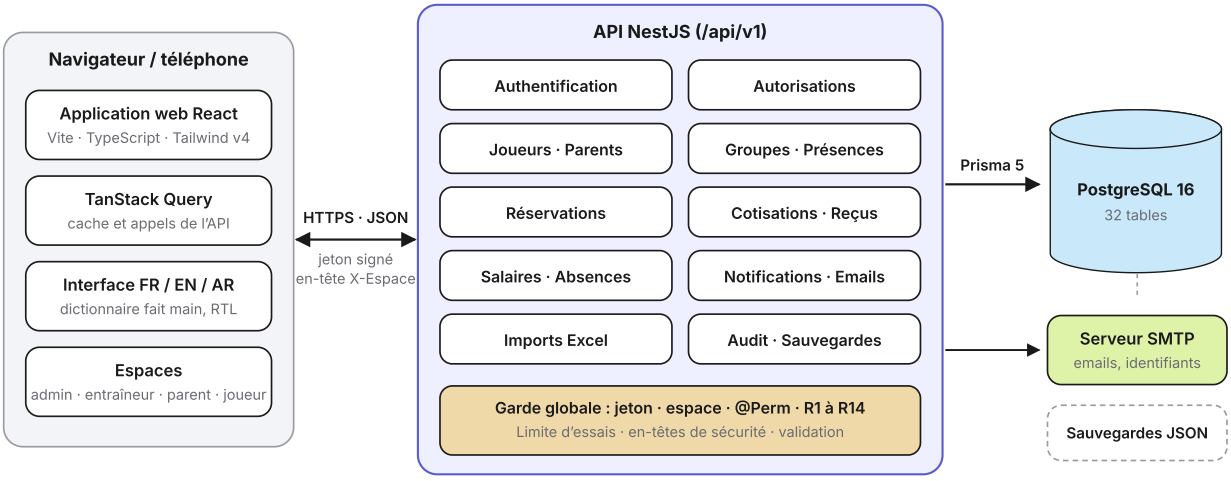


Figure 3 — Architecture de la plateforme

Chaque requête passe par une **garde globale** qui vérifie le jeton de session, l'état du compte, l'espace choisi (en-tête `X-Espace`) et, dans l'espace d'administration, l'autorisation requise par la route (décorateur `@Perm('payments.collect')` par exemple). Les autorisations des rôles du bureau sont stockées en base et réglées par le président.

3.2 Modèle de données

La base compte **32 tables** et **20 énumérations**, décrites dans un schéma Prisma et créées par **14 migrations** versionnées. La figure suivante présente les entités principales (les attributs techniques sont omis).

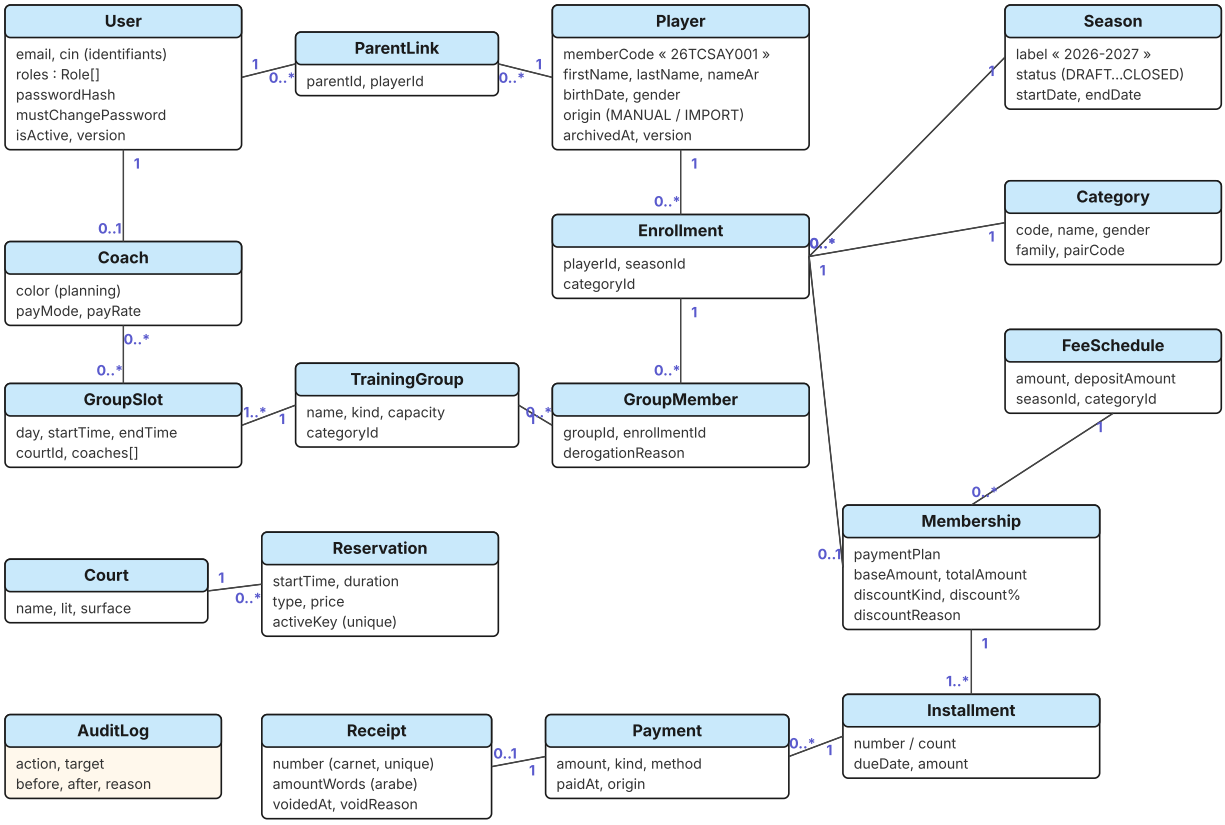


Figure 4 — Diagramme de classes simplifié

- un **User** (compte) peut être joueur, parent, entraîneur et membre du bureau à la fois ; parents et joueurs sont liés par **ParentLink** (plusieurs-à-plusieurs) ;
- un joueur a une **inscription** (Enrollment) par saison, dans une catégorie ; l'inscription le place dans un ou plusieurs groupes (**GroupMember**) ;
- la **cotisation** (Membership) est créée à partir du tarif ; elle porte le mode de paiement et l'éventuelle réduction, et se découpe en **tranches** (Installment) ;
- chaque encaissement (**Payment**) peut recevoir un **reçu** (Receipt) dont le numéro, pré-imprimé sur le carnet, est unique et jamais réutilisé ;
- la **réservation** porte une clé unique terrain + heure de début, ce qui empêche la double réservation au niveau de la base ;
- le **journal d'audit** (AuditLog) enregistre qui, quand, avant, après et motif ; un déclencheur PostgreSQL y interdit toute modification ou suppression.

3.3 Scénarios principaux

Authentification

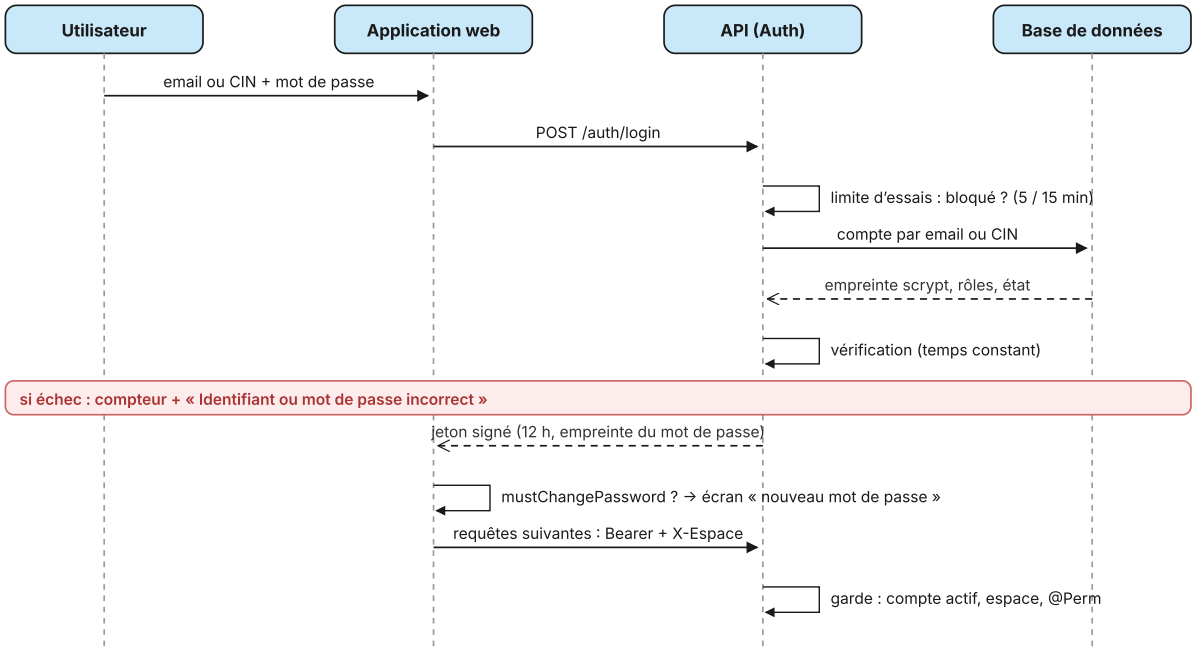


Figure 5 — Diagramme de séquence : connexion

Encaissement et reçu

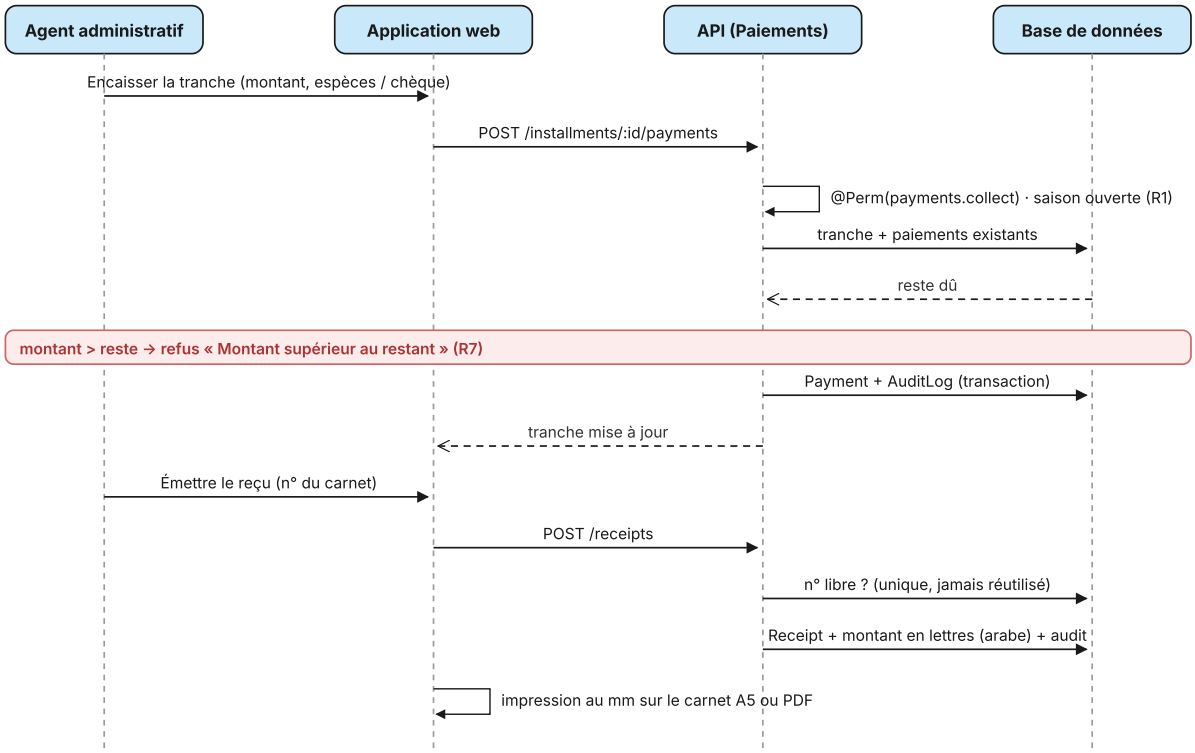


Figure 6 — Diagramme de séquence : encaissement d'une tranche et émission du reçu

Réservation d'un terrain

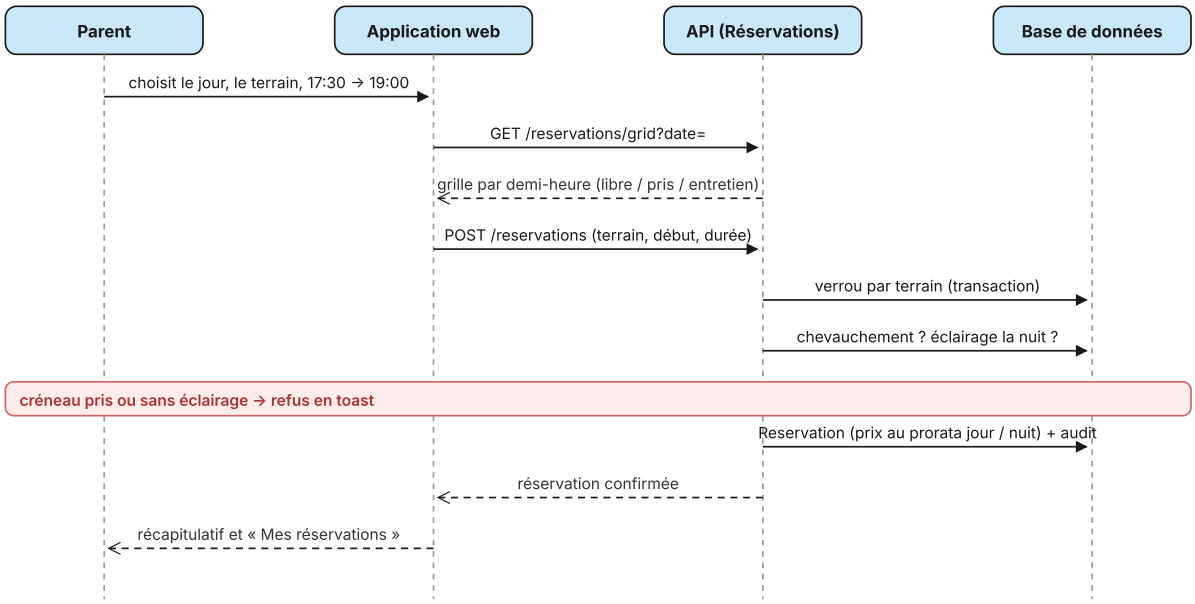


Figure 7 — Diagramme de séquence : réservation d'un terrain

3.4 Choix de conception

- **Règles côté serveur** : l'interface n'est qu'un confort ; une règle contournée dans le navigateur reste refusée par l'API.
- **Peu de dépendances** : client SMTP, lecteur Excel, jeton de session et limite d'essais ont été écrits sans bibliothèque supplémentaire, pour limiter la surface d'attaque et garder la maîtrise du code.
- **Historique conservé** : archivage plutôt que suppression, paiements et reçus jamais modifiés, corrections par écritures motivées.
- **Données importées** : origine IMPORT conservée, aperçu systématique avant enregistrement, re-import sans doublon.

4. Réalisation

4.1 Environnement de développement

Couche	Technologies
Interface web	React 19, Vite, TypeScript, Tailwind CSS v4, React Router, TanStack Query ; composants écrits à la main (aucune bibliothèque de composants).
API	NestJS 11 (Node.js, TypeScript), validation class-validator, documentation Swagger.
Données	PostgreSQL 16, Prisma 5 (schéma, migrations, client typé).
Emails	Client SMTP fait main (TLS, STARTTLS), file d'envoi en arrière-plan.
Outils	VS Code, Git / GitHub, Docker (PostgreSQL, Mailpit), pgAdmin, Playwright (tests d'écrans), Render (démonstration en ligne).

Tableau 5 — Environnement technique

Mesure	Valeur
Lignes de code TypeScript (API / web)	≈ 10 700 / ≈ 12 800
Routes de l'API	142, réparties dans 31 contrôleurs
Écrans web	40 pages, 22 composants d'interface
Tables / migrations	32 tables, 14 migrations
Traductions	≈ 330 libellés en anglais et en arabe

Tableau 6 — Quelques chiffres du projet

4.2 Connexion et espaces

Les comptes sont créés par le club : l'utilisateur reçoit par email un identifiant et un mot de passe temporaire, qu'il doit changer à la première connexion. On se connecte par email ou par numéro de CIN.

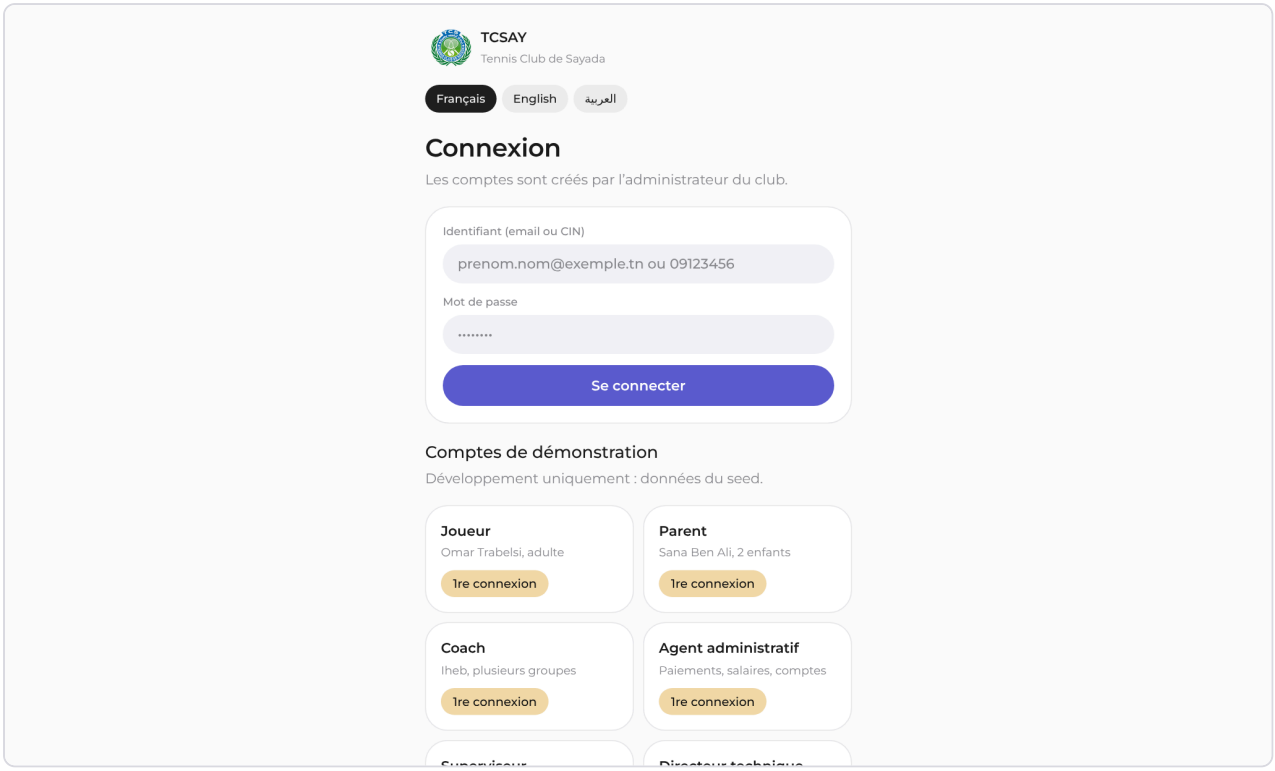


Figure 8 — Écran de connexion

4.3 Espace administration

Tableau de bord et planning

Le tableau de bord résume la saison : joueurs actifs, montants encaissés, tranches en retard, réservations, tranches à relancer. Le planning affiche, jour par jour, l'occupation des terrains par les groupes (couleur de l'entraîneur) et par les réservations.

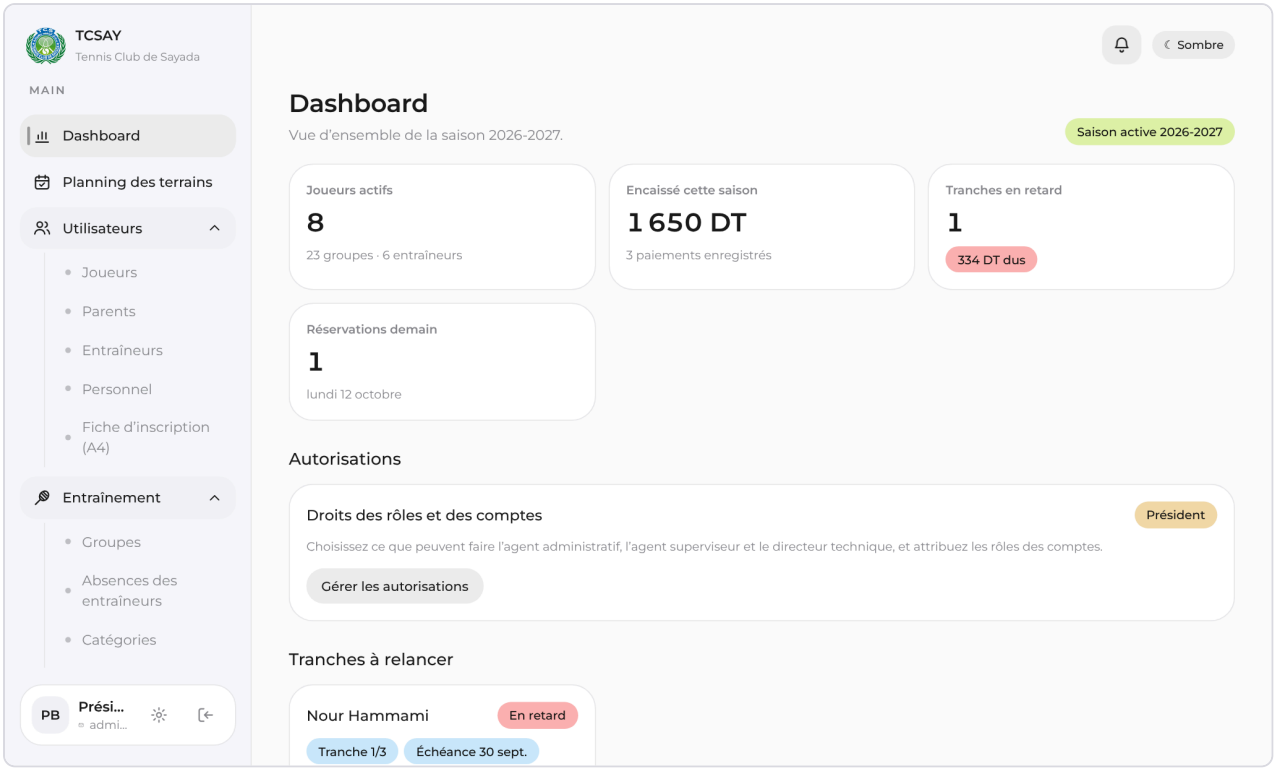


Figure 9 — Tableau de bord

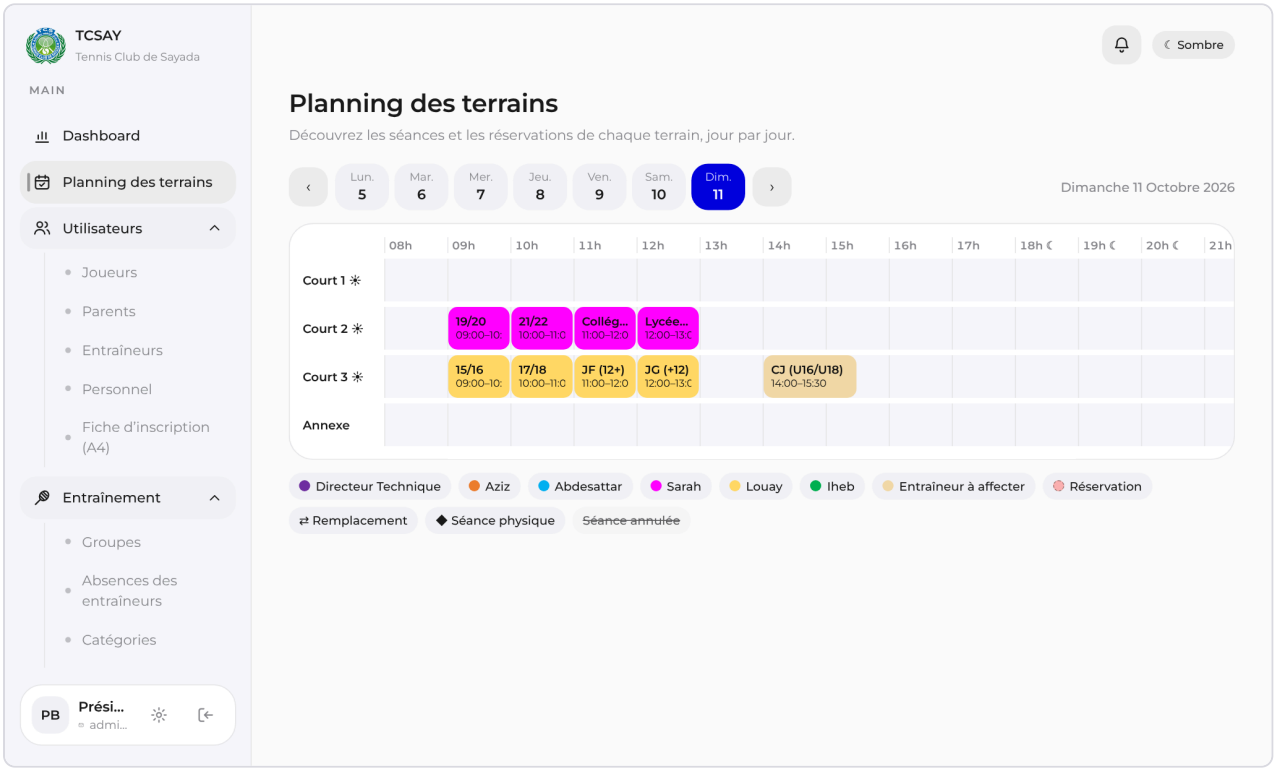


Figure 10 — Planning des terrains

Joueurs

Chaque joueur reçoit un code club (« 26TCSAY001 » : année d'inscription et numéro). La liste se filtre par catégorie, se trie (nom, âge, catégorie, groupe, code) et s'affiche en cases ou en

liste. Des pastilles signalent ce qui reste à compléter : date de naissance, email du parent, cotisation.

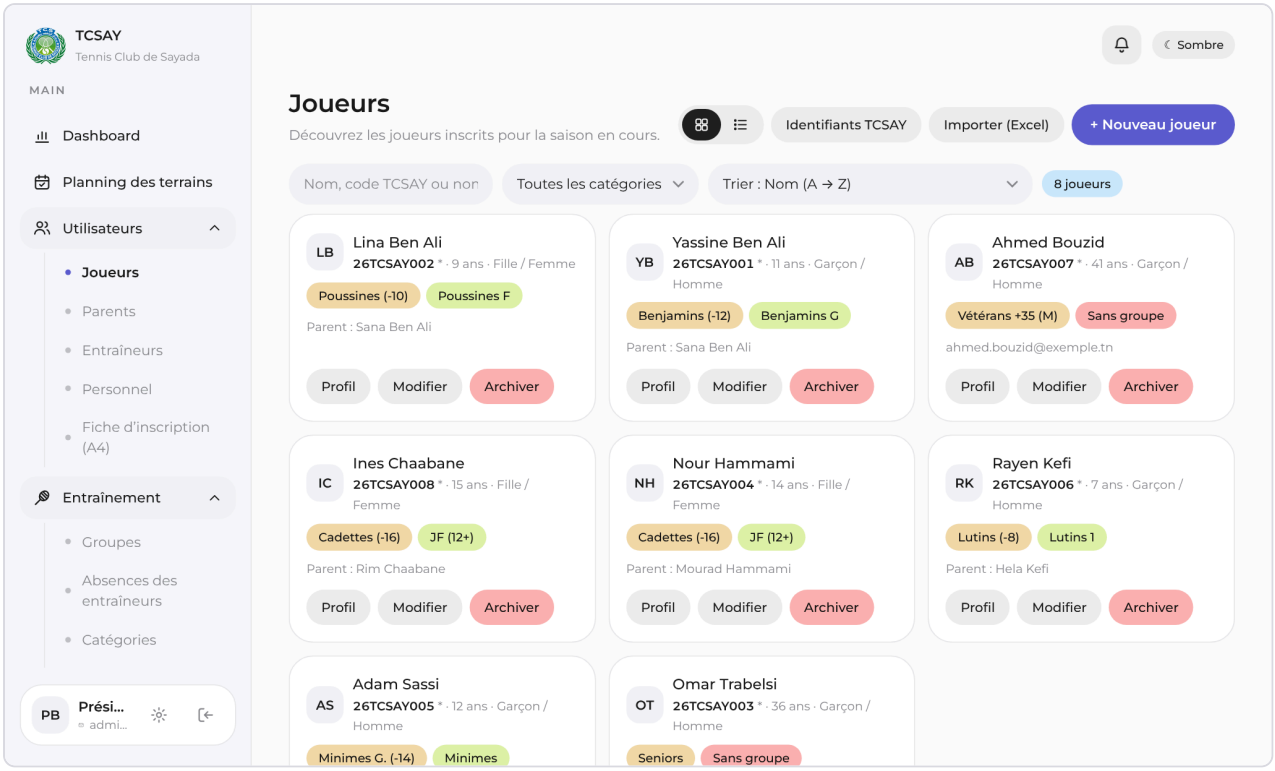


Figure 11 — Liste des joueurs (affichage en cases)

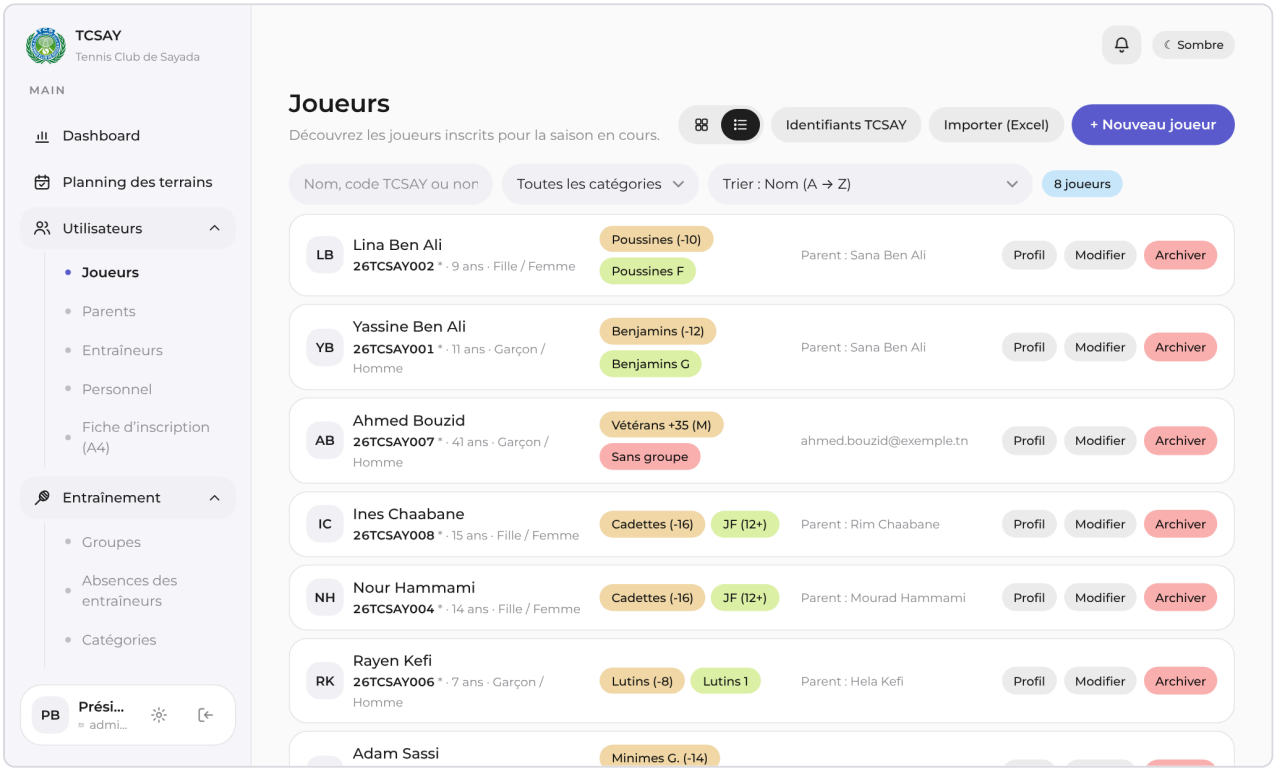


Figure 12 — Liste des joueurs (affichage en liste)

Dans la fiche « Modifier », l'administrateur choisit le mode de paiement et applique une réduction (fratrie, spéciale ou exonération) : le montant à payer est recalculé en direct, les

frères et sœurs inscrits sont signalés et le motif est obligatoire.

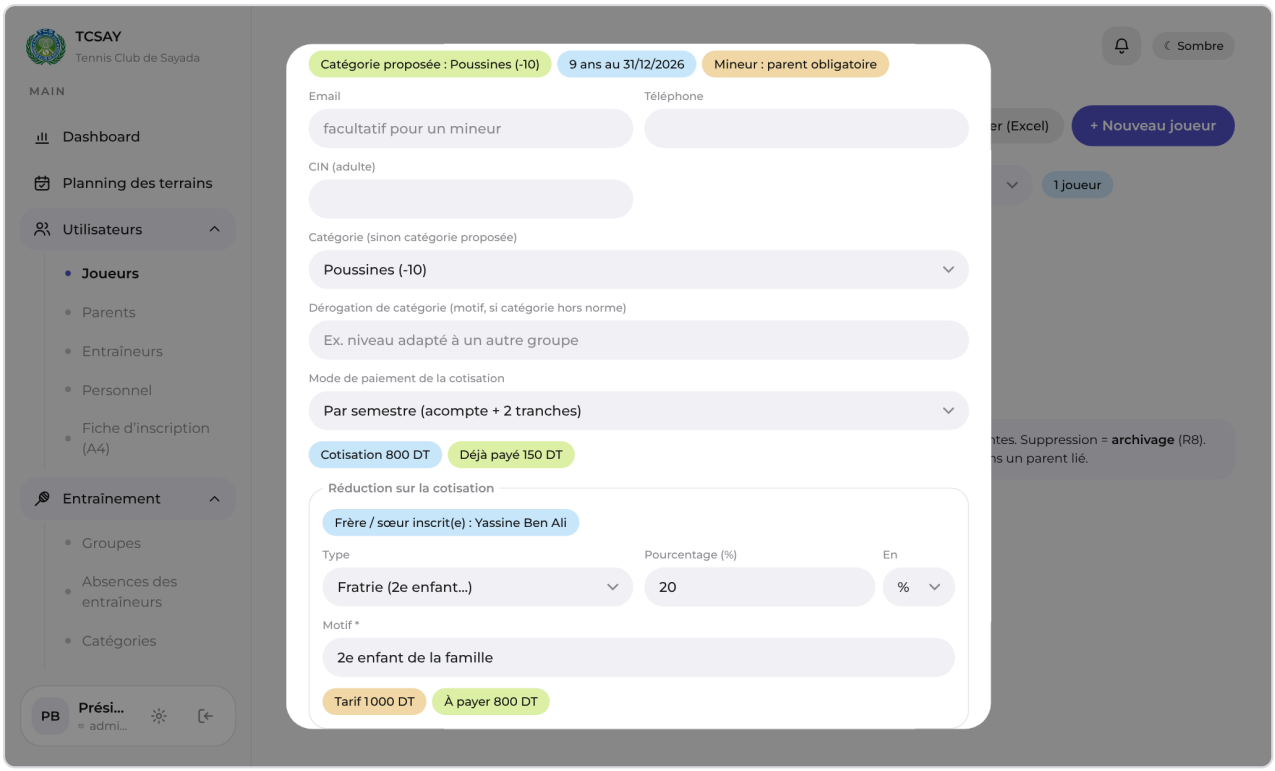


Figure 13 — Modification d'un joueur : mode de paiement et réduction

Groupes

Les cartes des groupes montrent les horaires et les entraîneurs. La fiche d'un groupe détaille les joueurs (code, âge, parents, assiduité), les dernières séances, et permet de chercher ou de créer un joueur pour l'ajouter, en respectant la capacité et la catégorie (R4).

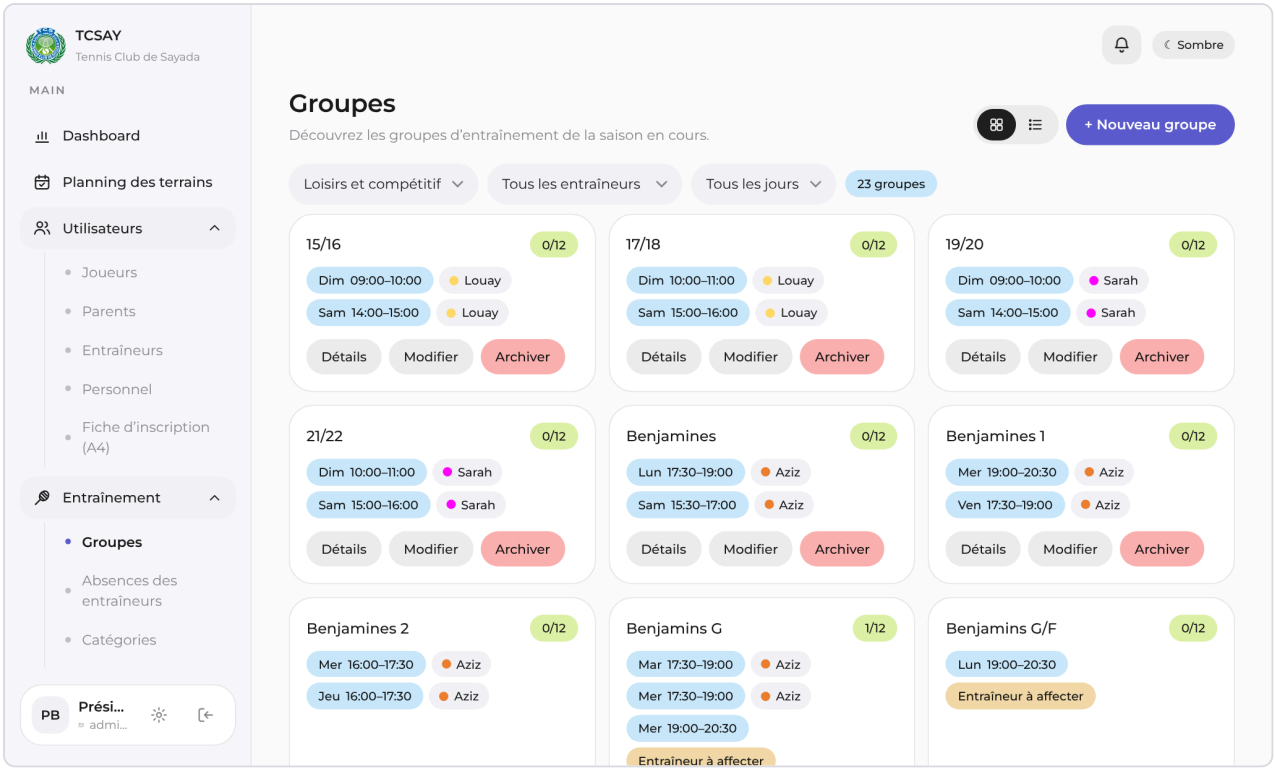


Figure 14 — Groupes d'entraînement

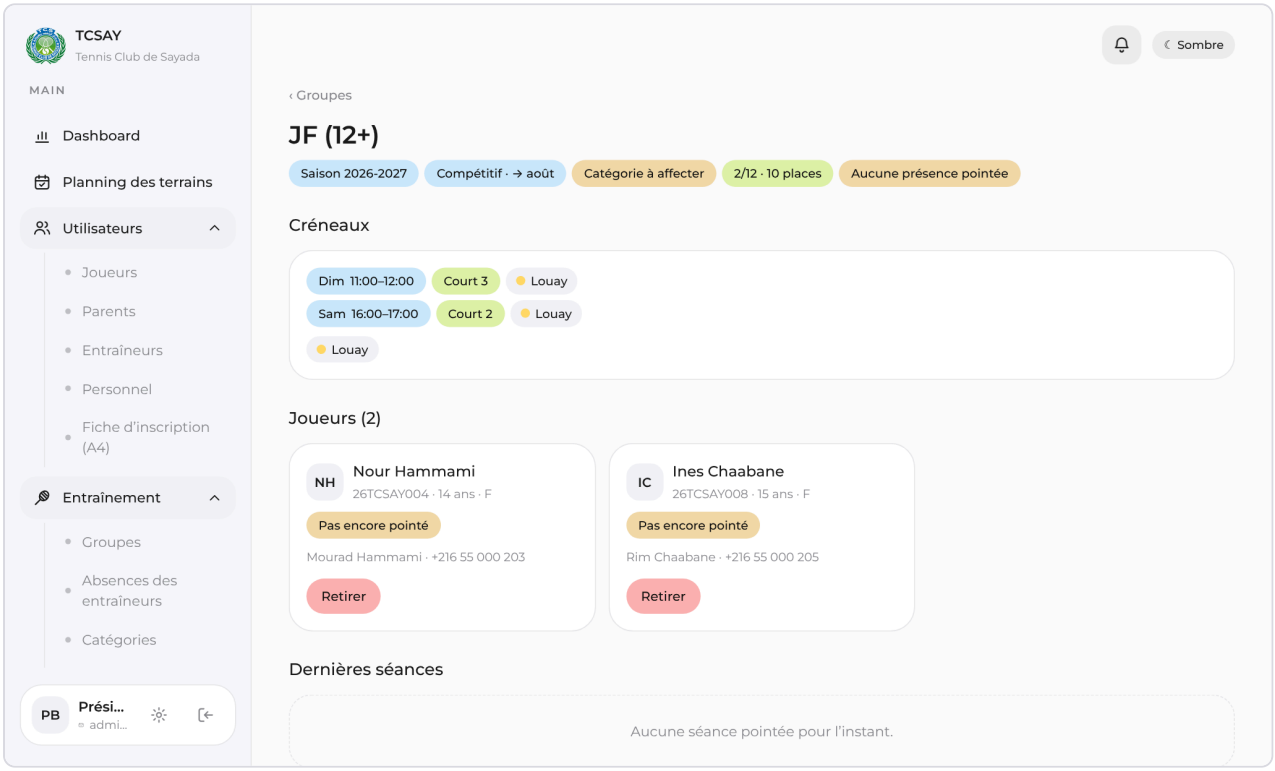


Figure 15 — Fiche d'un groupe

Paiements et reçus

Les tranches de chaque joueur s'affichent en vert (payée) ou en rose (à payer, en retard). L'encaissement crée un paiement non modifiable (R7) ; le reçu est émis avec le numéro pré-

imprimé du carnet, puis imprimé au millimètre près sur le carnet A5 ou enregistré en PDF. Le montant est écrit en lettres en arabe.

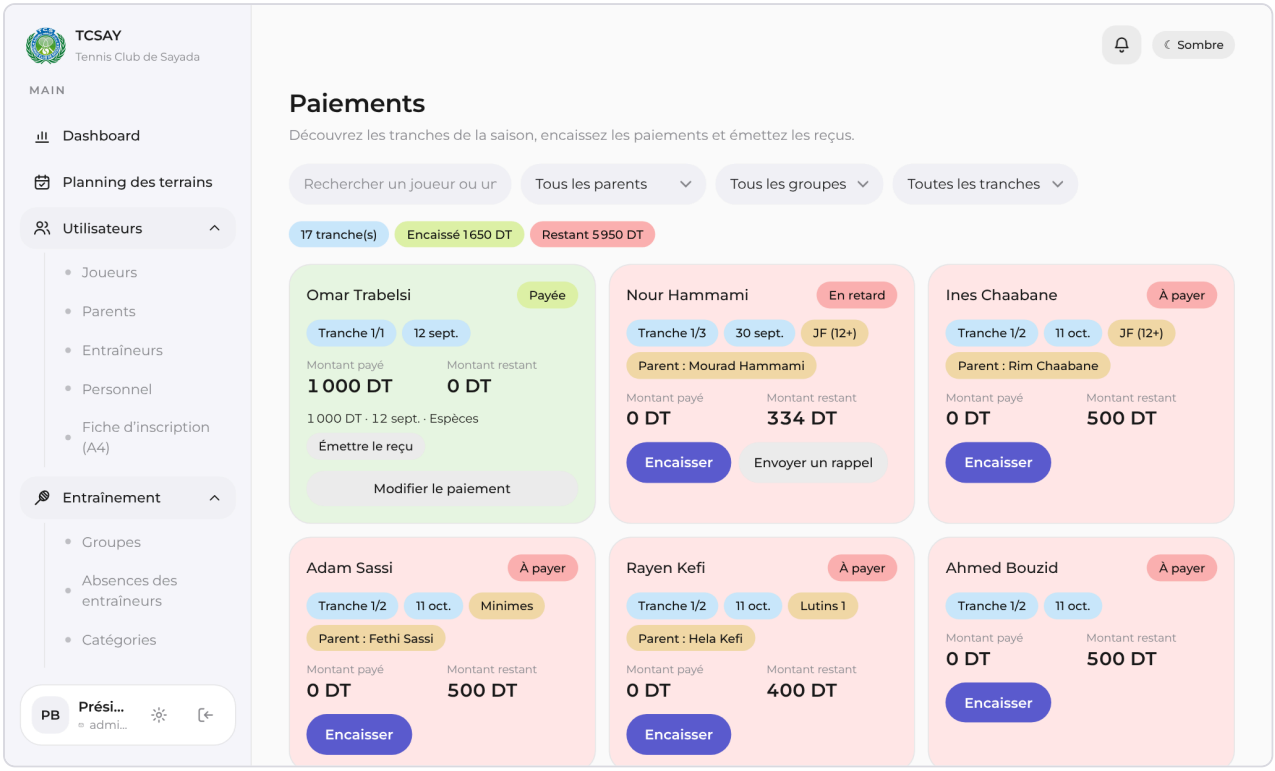


Figure 16 — Paiements des joueurs

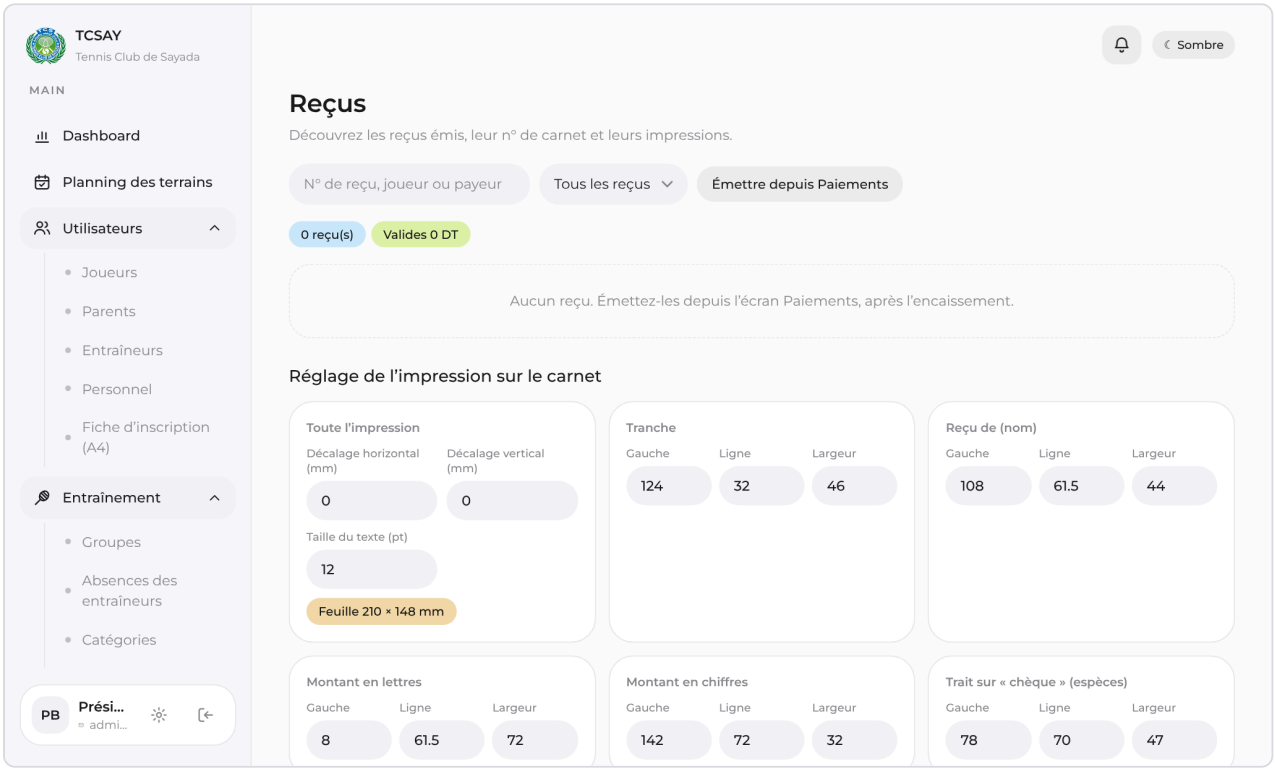


Figure 17 — Reçus et réglage de l'impression sur le carnet

Réservations

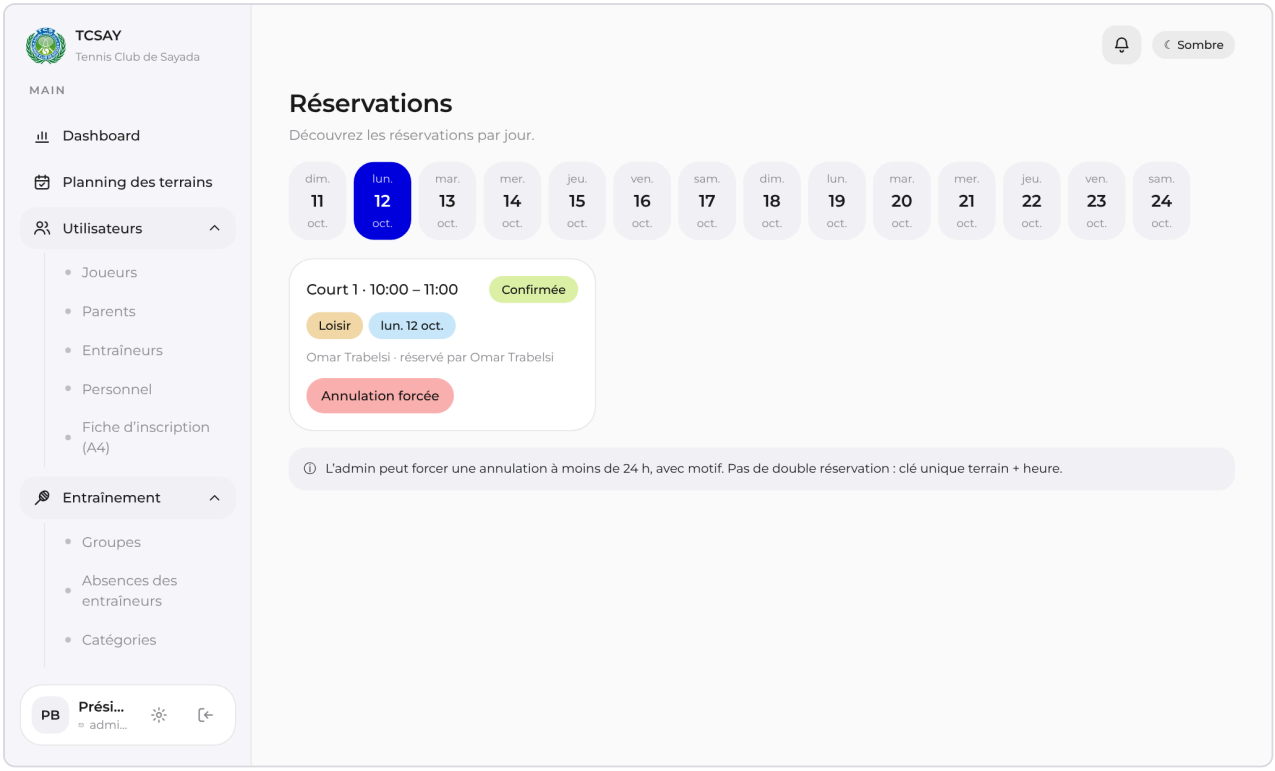


Figure 18 — Réservations (vue administration)

Notifications, imports et autorisations

Le bureau envoie des messages aux parents, aux joueurs, aux entraîneurs, à un groupe ou à des personnes choisies ; ils apparaissent dans la cloche de l'application (et par email au choix), avec un suivi du nombre de lectures.

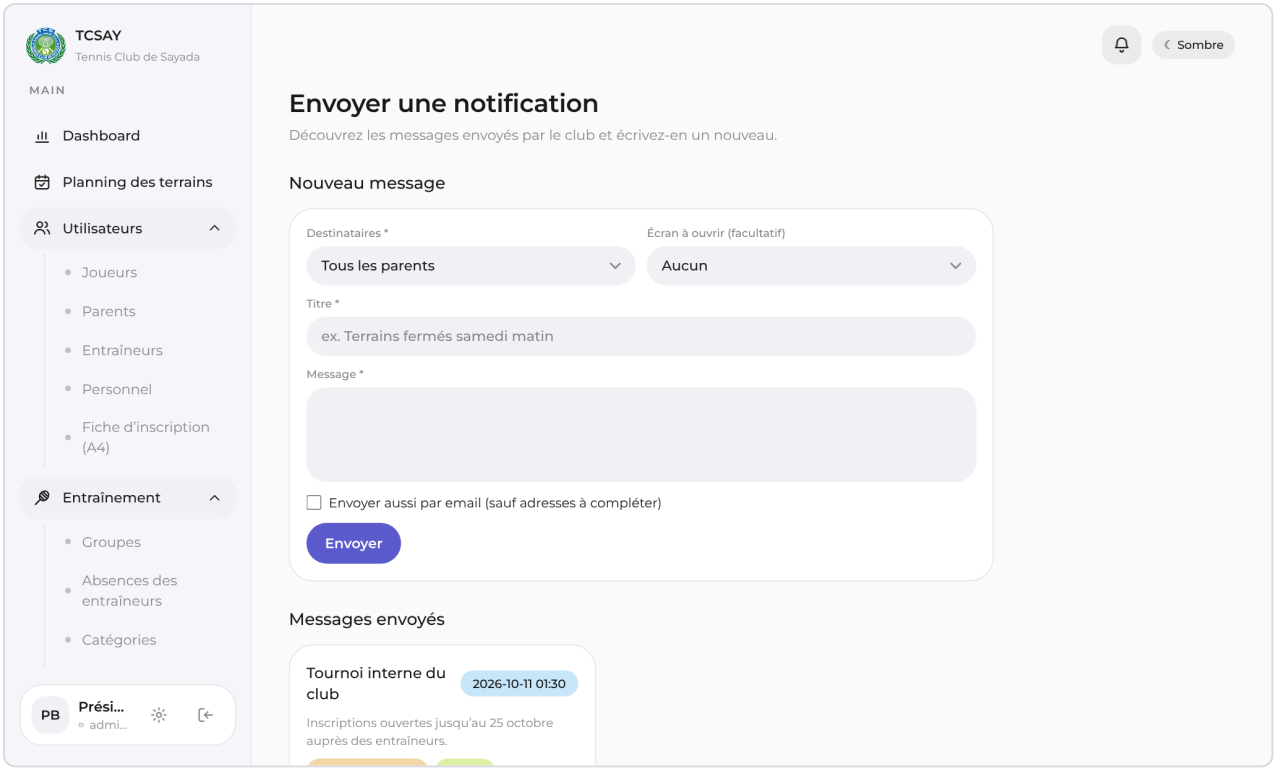


Figure 19 — Envoi d'une notification

Les listes Excel existantes sont importées avec un aperçu ligne par ligne (prête, à vérifier, ignorée) avant tout enregistrement. Les noms sont reconnus malgré les fautes d'orthographe, mais le prénom doit correspondre, pour éviter de confondre deux enfants d'une même famille.

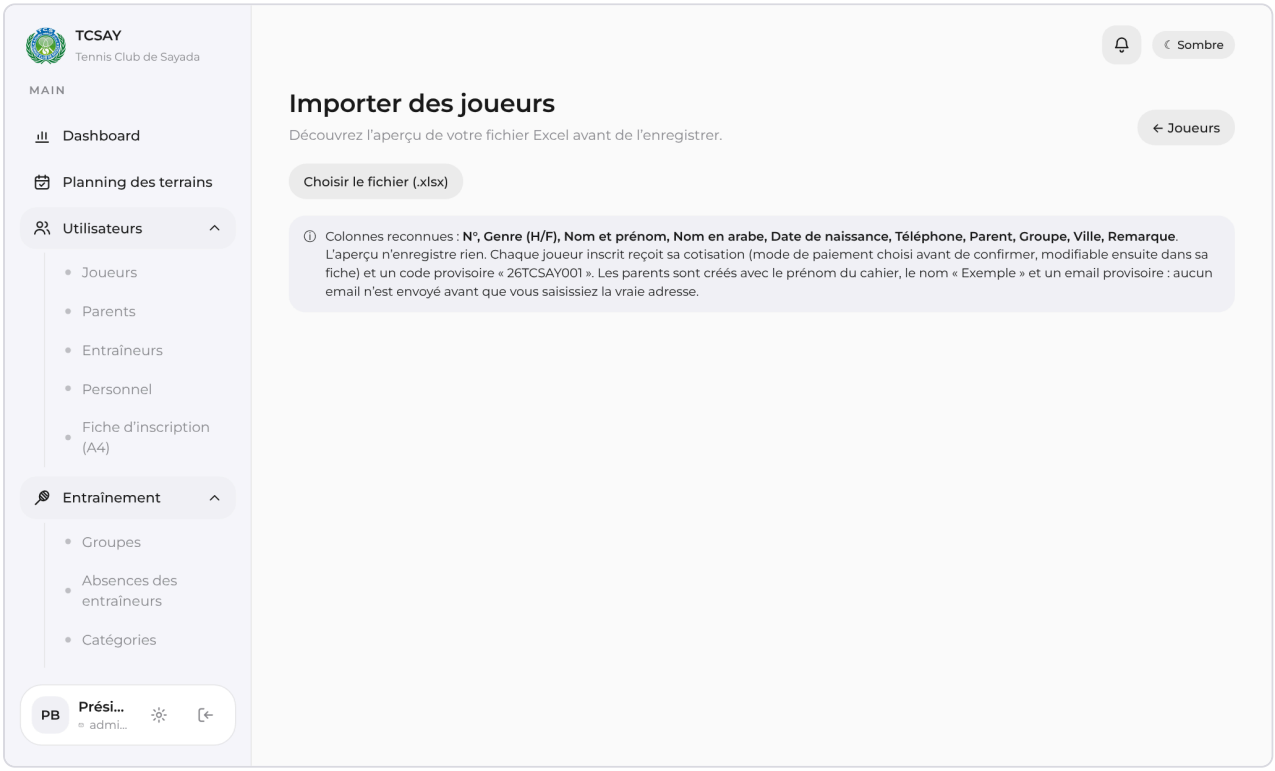


Figure 20 — Import des joueurs depuis Excel

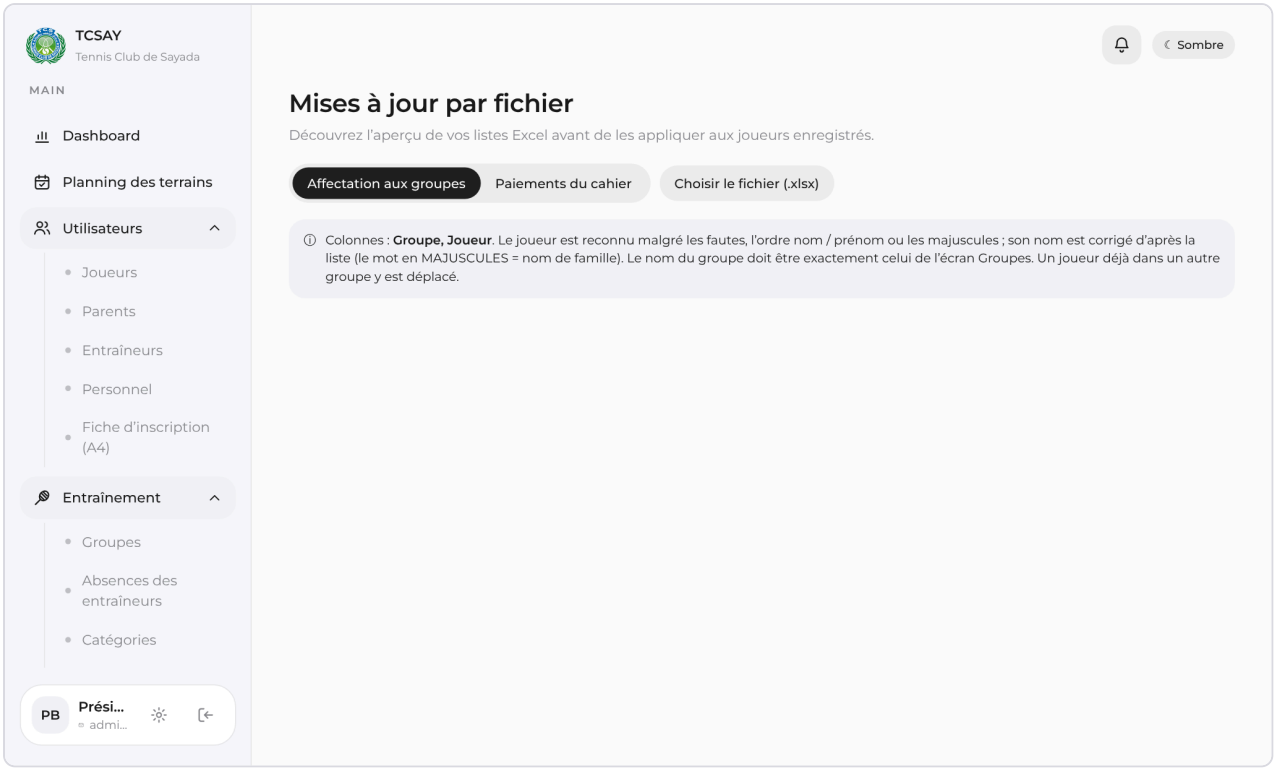


Figure 21 — Mises à jour par fichier : groupes et paiements du cahier

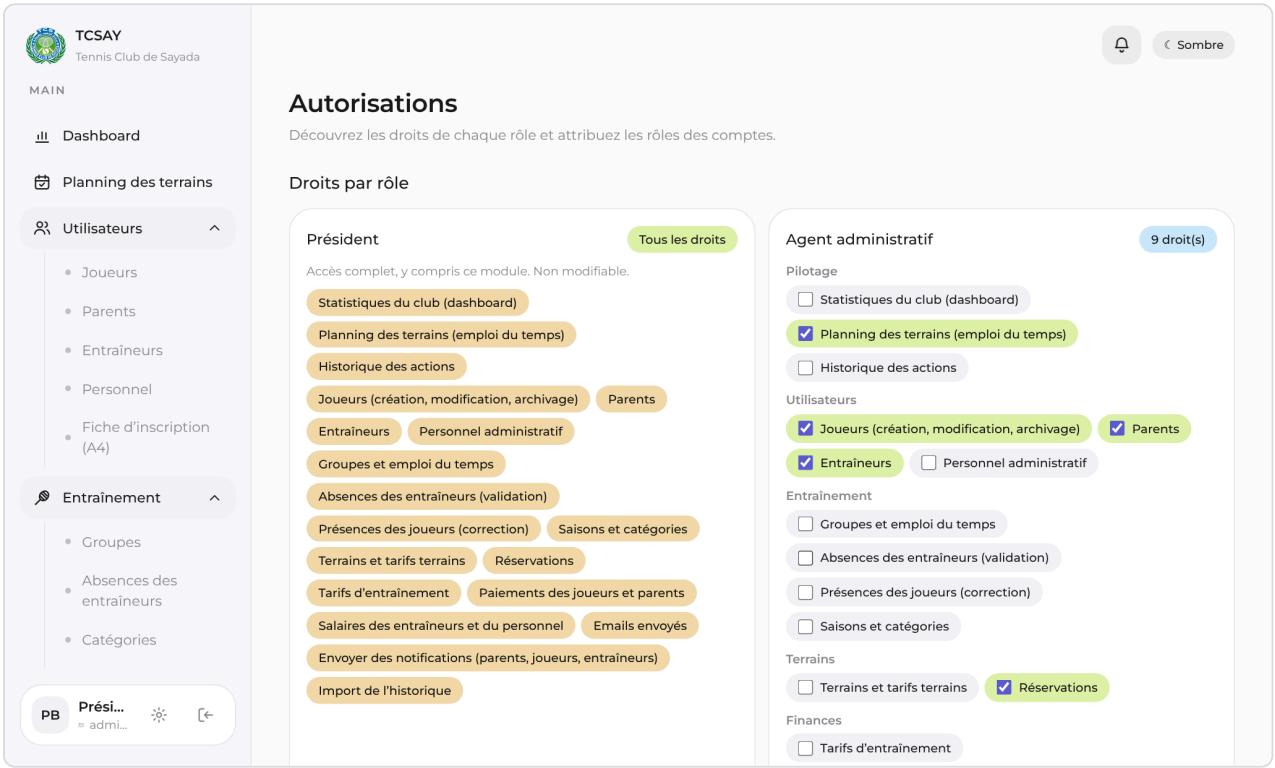


Figure 22 — Autorisations réglées par le président

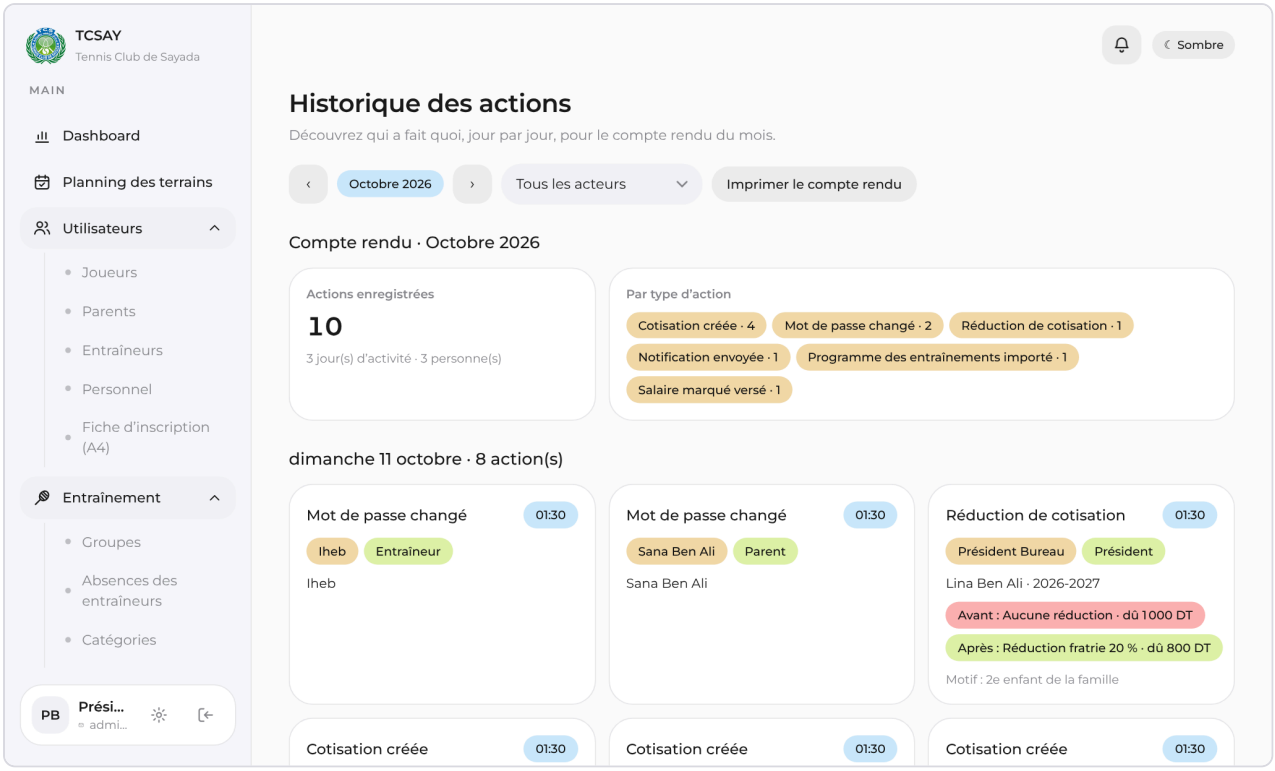


Figure 23 — Historique des actions (journal d'audit)

4.4 Espace parent et joueur

L'accueil présente la prochaine séance de l'enfant choisi, ses paiements, ses réservations et les événements du club. Les paiements sont détaillés par tranche ; la réservation se fait sur une grille par demi-heure (départs à l'heure ou à la demi-heure, de 1 h à 4 h), avec le prix calculé au prorata jour / nuit.

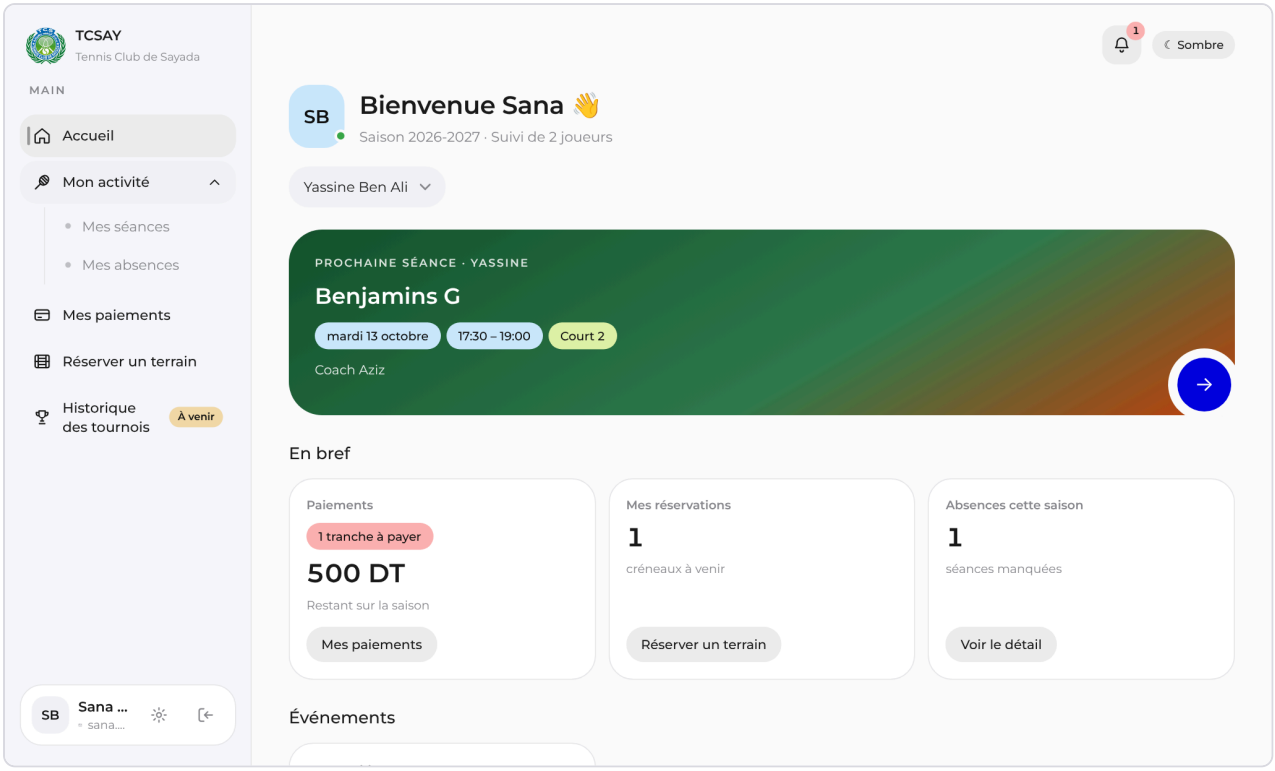


Figure 24 — Accueil du parent

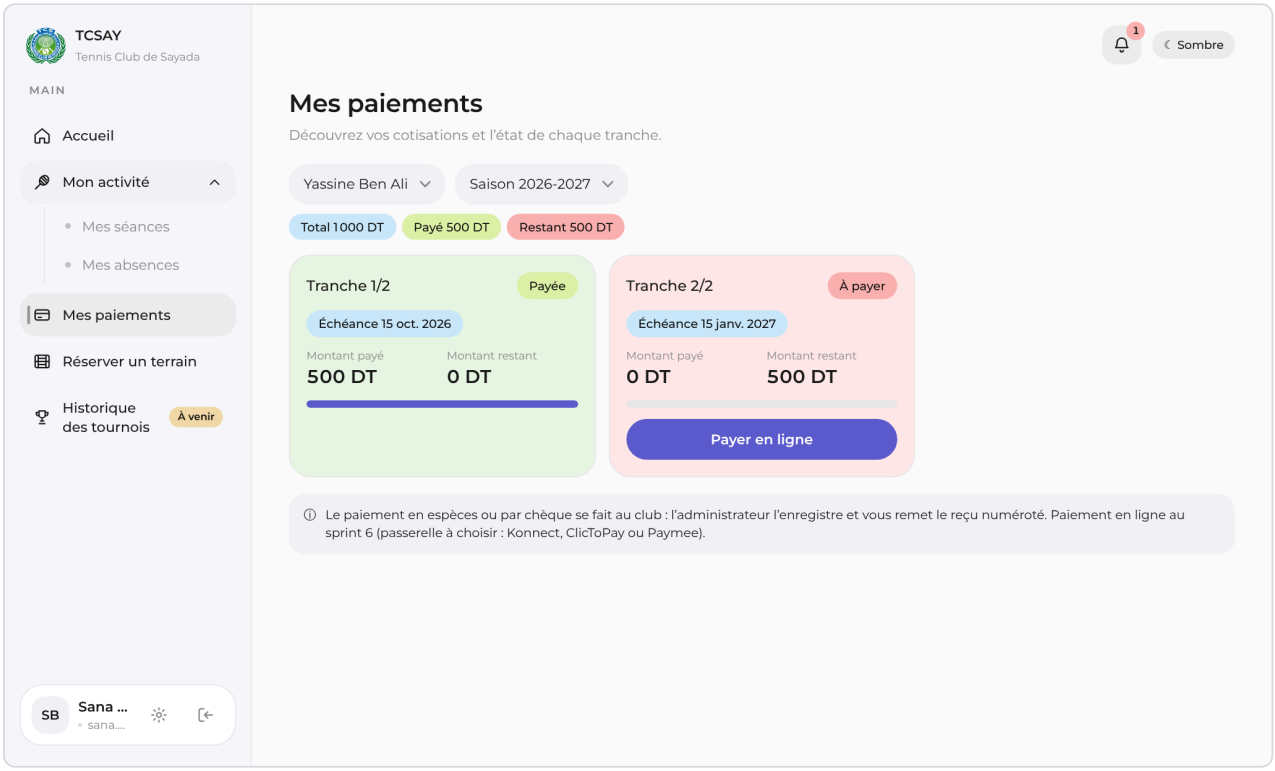


Figure 25 — Paiements par tranches (parent)

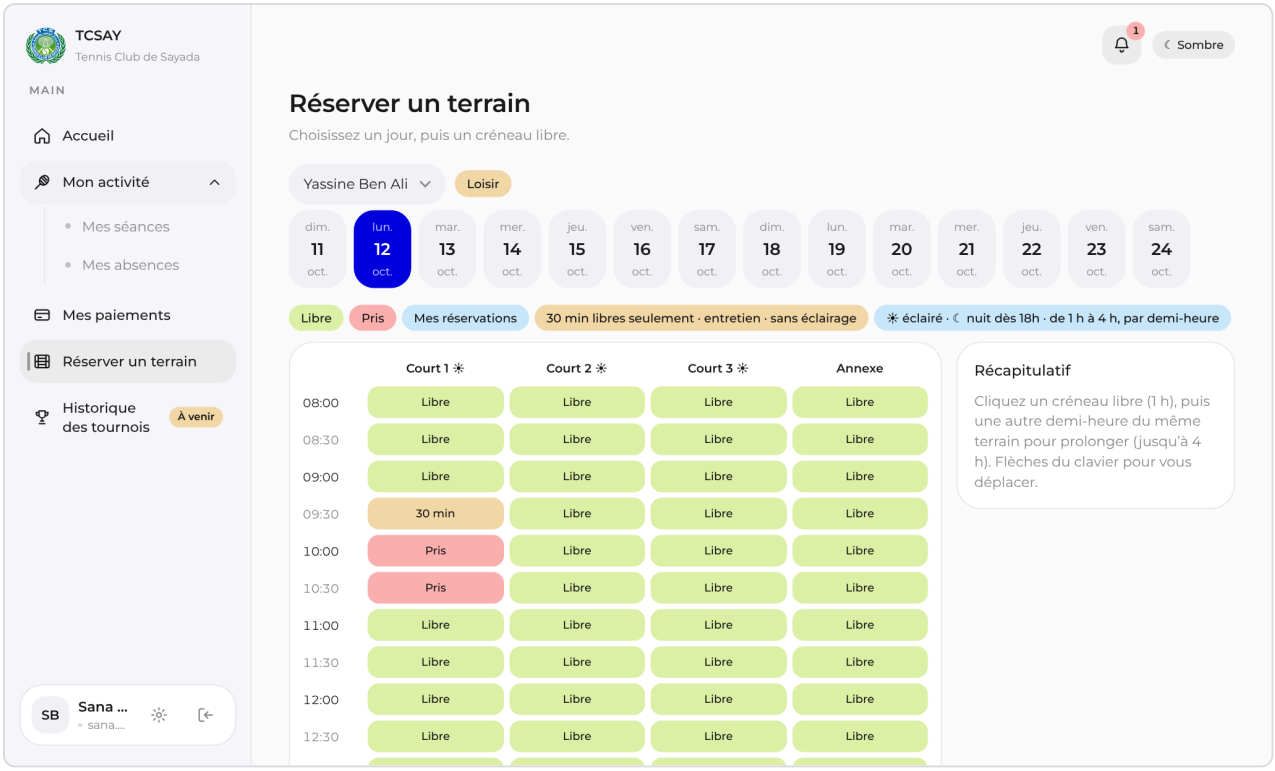


Figure 26 — Réservation d'un terrain par demi-heure

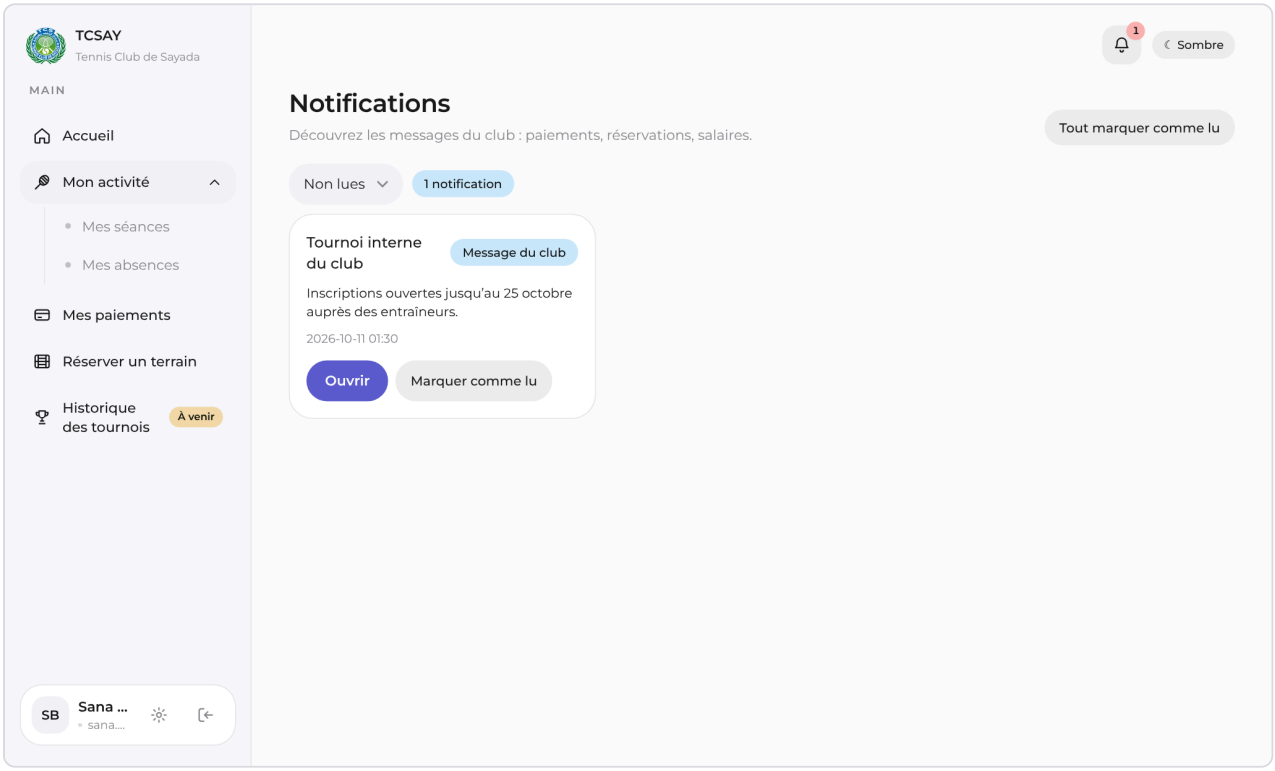


Figure 27 — Notifications reçues

4.5 Espace entraîneur

L'entraîneur voit son calendrier de séances, pointe les présences de la séance en cours, déclare ses absences et consulte ses salaires (estimations et salaires versés).

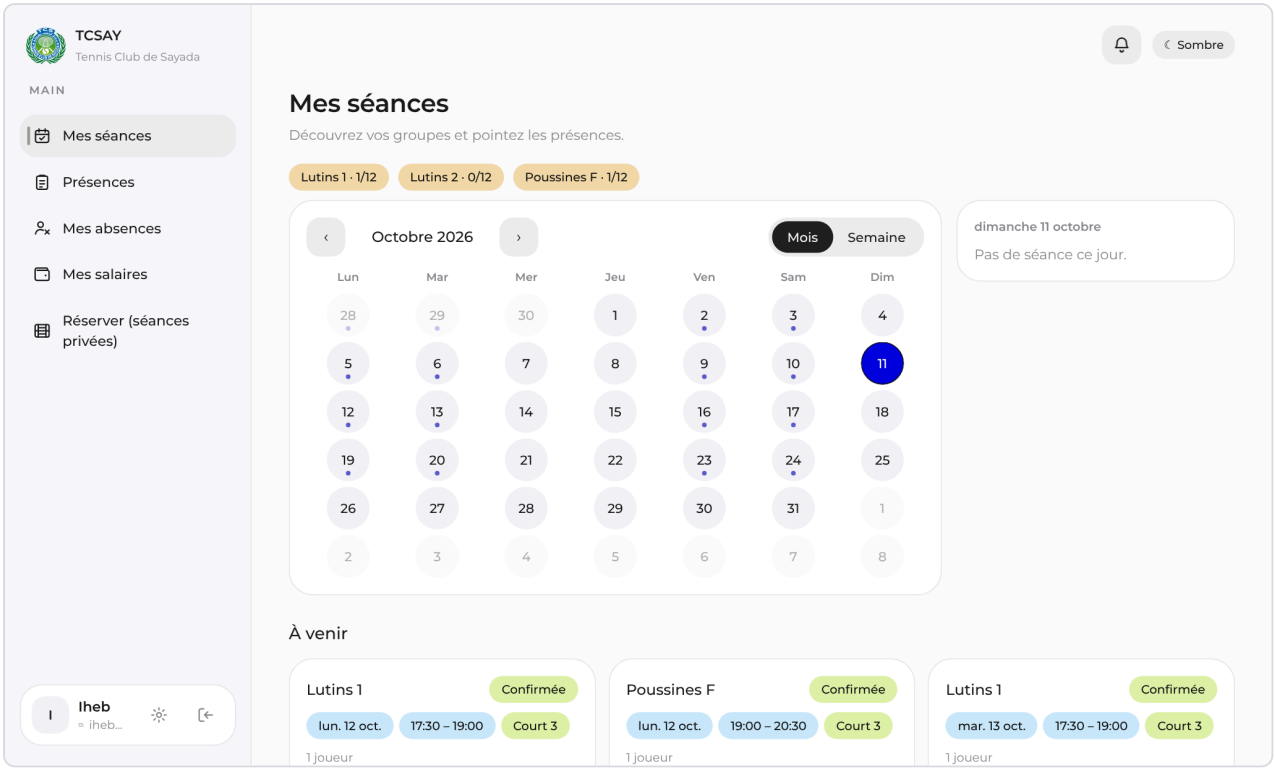


Figure 28 — Séances de l'entraîneur

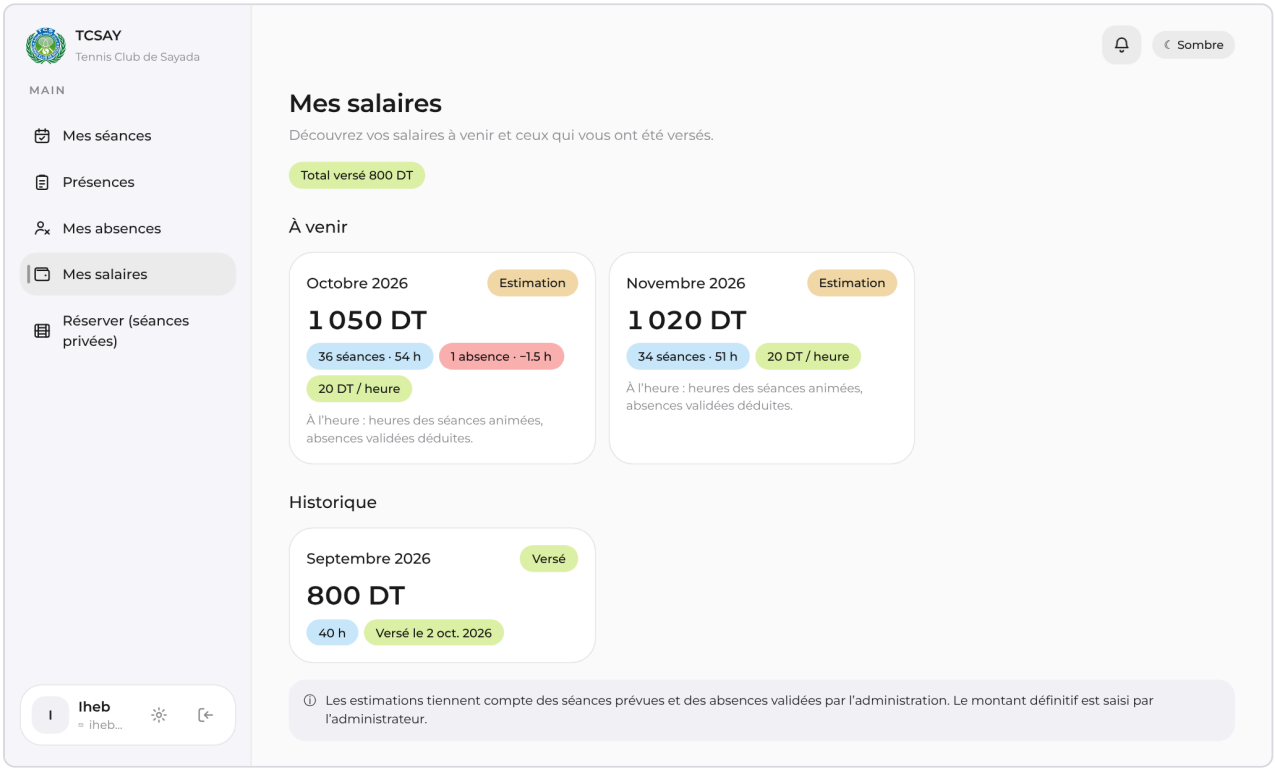


Figure 29 — Salaires de l'entraîneur

4.6 Langues et téléphone

L'interface existe en français, anglais et arabe ; en arabe, toute la mise en page passe de droite à gauche. Sur téléphone, les cartes s'empilent sur une colonne et le menu devient un tiroir.

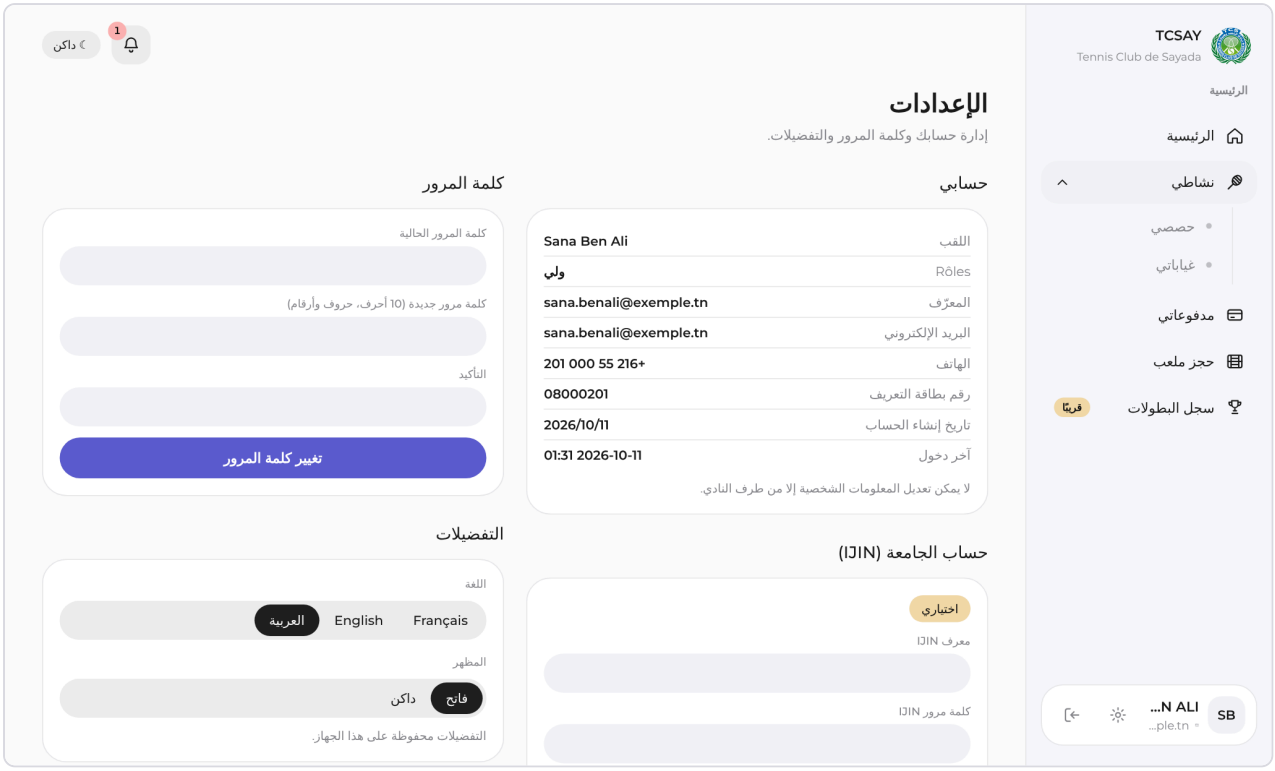


Figure 30 — Paramètres en arabe (de droite à gauche)

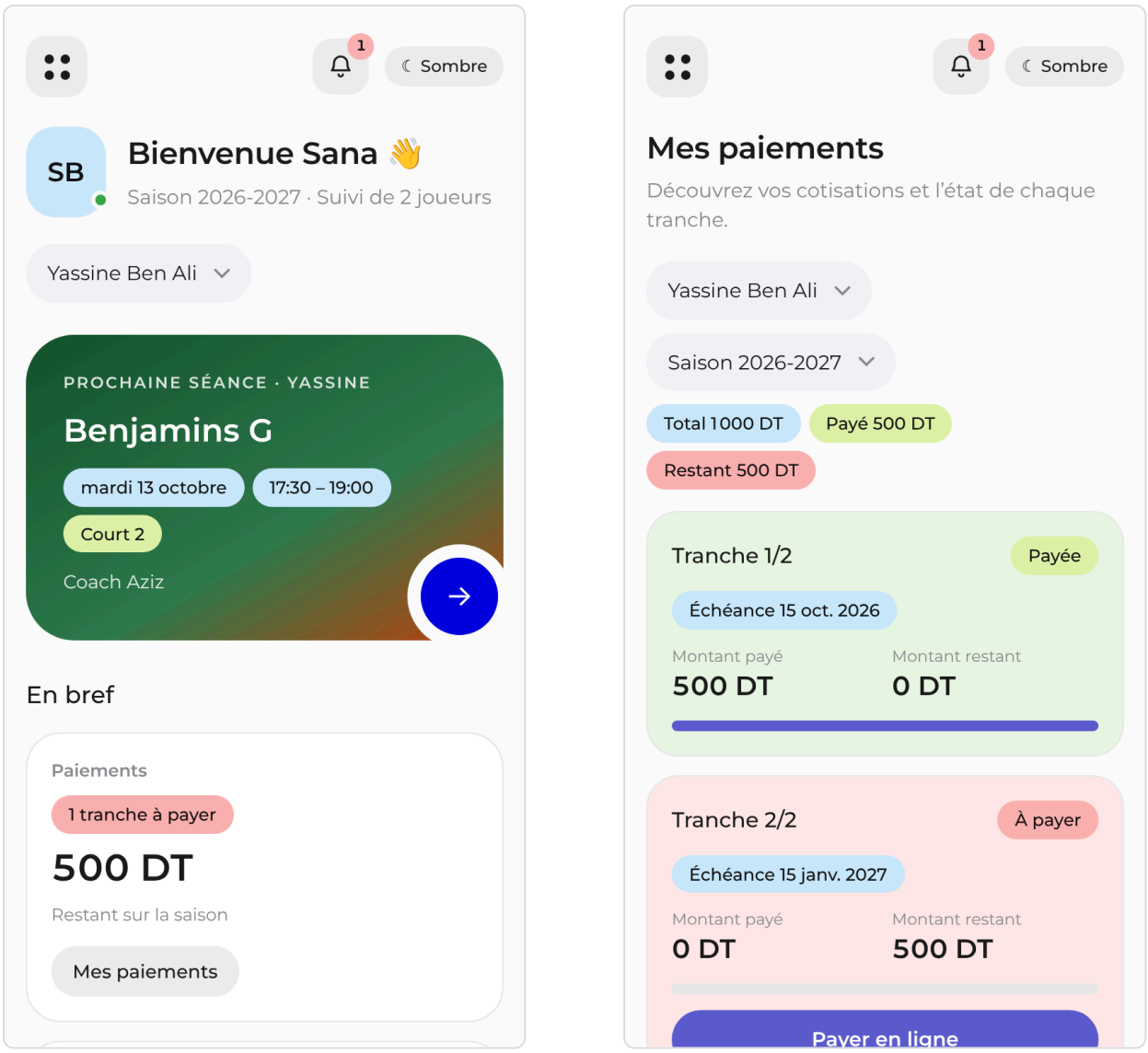


Figure 31 — Affichage sur téléphone : accueil et paiements

5. Sécurité, tests et déploiement

5.1 Sécurité

Mesure	Mise en œuvre
Mots de passe	Empreinte scrypt avec sel ; 10 caractères au moins, lettres et chiffres, ni nom, ni email, ni mot courant ; changement obligatoire du mot de passe temporaire.
Sessions	Jeton signé HS256 valable 12 h, contenant une empreinte du mot de passe : changer de mot de passe déconnecte les autres appareils.
Essais de connexion	5 échecs par identifiant (20 par adresse) en 15 minutes → blocage 15 minutes ; même temps de réponse pour un compte inexistant.
Droits	Garde globale : compte actif, espace, rôle, autorisation de la route ; données filtrées (un parent ne voit que ses enfants).
HTTP	Politique de contenu (CSP), interdiction d'intégration dans un autre site, HSTS, CORS fermé, réponses de l'API jamais mises en cache, 300 requêtes par minute et par adresse.
Données saisies	Validation de chaque champ ; tout champ inconnu est refusé.
Secrets	Variables d'environnement hors du dépôt ; mots de passe IJIN chiffrés en AES-GCM.
Traçabilité	Journal d'audit en lecture seule, protégé par un déclencheur PostgreSQL.

Tableau 7 — Mesures de sécurité

5.2 Protection des données de la base

- base de démonstration et base de production **séparées** ; la base de production est marquée et refuse tout chargement de données de démonstration ;
- chaque changement de structure est annoncé, demande une confirmation et est précédé d'une **sauvegarde automatique** ;
- sauvegardes complètes (JSON et pg_dump) avec rotation, copiables vers un dossier synchronisé hors du PC, et restauration testée dans une base neuve ;
- les migrations n'effacent jamais de données : elles ajoutent des colonnes ou des tables.

5.3 Tests

Chaque fonctionnalité a été testée de bout en bout : appels de l'API avec des scénarios automatisés (succès, refus attendus) et parcours des écrans dans un navigateur automatisé (Playwright), y compris en largeur téléphone. Quelques exemples :

Scénario	Résultat attendu	Obtenu
6 mauvais mots de passe	5 refus puis blocage 15 min	✔
Mot de passe « azerty12345 »	Refus : trop courant	✔
Ancienne session après changement de mot de passe	Refusée (401)	✔
Jeton falsifié « alg : none »	Refusé	✔
Coach qui lit les cotisations	Refusé (403)	✔
Encaissement supérieur au reste dû	Refus R7	✔
Réduction après paiement sans motif	Refus R7	✔
Suppression du seul parent d'un mineur	Refus R9	✔
Double réservation d'un même créneau	Refusée	✔
Re-import du même fichier de paiements	Aucun doublon (n° de reçu)	✔
« Benjamines 2 » dans un fichier	Jamais confondu avec « Benjamines 1 »	✔ (après correction)

Tableau 8 — Exemples de scénarios de test

5.4 Déploiement

Une version de **démonstration** est hébergée sur Render : un service qui sert l'API et le site compilé, et une base PostgreSQL contenant uniquement des données fictives, avec des comptes de démonstration pour les collègues testeurs. Le déploiement est **manuel** : un envoi sur GitHub ne met rien en ligne sans validation.

Pour la mise en production, il est prévu : un nom de domaine au nom du club, un hébergement payant avec base de données sauvegardée chaque jour, HTTPS, des secrets générés aléatoirement, puis un essai d'une semaine avec quelques parents et un entraîneur avant l'ouverture à tous.

Conclusion générale et perspectives

Ce stage m'a permis de mener un projet complet, de l'analyse des besoins d'une vraie association jusqu'à une application utilisable : comptes et rôles, groupes et planning, réservations, cotisations et réductions, reçus imprimés sur le carnet du club, présences, salaires, notifications et import des listes existantes, le tout en trois langues et sur téléphone.

Difficultés rencontrées

- traduire des habitudes papier en règles précises (reçus pré-numérotés, réductions, cahier des paiements) ;
- reconnaître des noms mal orthographiés sans jamais confondre deux enfants (prénom obligatoire, groupes au nom exact) ;
- l'impression au millimètre sur un carnet pré-imprimé et le montant en lettres en arabe ;
- la sécurité et la protection des données de mineurs, et la séparation stricte entre données de démonstration et données réelles.

Compétences acquises

- conception d'une base relationnelle et de migrations sans perte de données ;
- développement d'une API REST sécurisée (NestJS, Prisma) et d'une interface React accessible et multilingue ;
- contrôle d'accès par rôles et par autorisations ; bonnes pratiques de sécurité web ;
- travail itératif avec un client réel, versionnement Git, rédaction de documentation.

Perspectives

- comptabilité du club : dépenses (électricité, eau, achats), bilan recettes / dépenses et exports Excel ;
- gestion du stock de matériel (balles, équipement remis aux entraîneurs) ;
- paiement en ligne via une passerelle tunisienne ;
- application mobile (d'abord installable depuis le navigateur, puis sur les magasins d'applications) ;
- numérisation des cahiers historiques depuis 2010 par reconnaissance d'écriture, avec validation humaine.

Webographie

- NestJS — Documentation officielle, https://docs.nestjs.com
- Prisma — Documentation officielle, https://www.prisma.io/docs
- PostgreSQL 16 — Documentation, https://www.postgresql.org/docs/16/
- React — Documentation, https://react.dev
- Tailwind CSS v4 — Documentation, https://tailwindcss.com/docs
- TanStack Query — Documentation, https://tanstack.com/query
- OWASP — Authentication Cheat Sheet, https://cheatsheetseries.owasp.org
- OWASP Top 10, https://owasp.org/www-project-top-ten/
- MDN Web Docs — Content-Security-Policy, HTTP Strict Transport Security, https://developer.mozilla.org
- RFC 7519 — JSON Web Token (JWT)
- Conventional Commits 1.0.0, https://www.conventionalcommits.org

Annexes

Annexe A — Extrait : règles du mot de passe

```
/** Mots de passe trop courants, refusés quelle que soit leur longueur. */
const COMMON = [
  'password', 'motdepasse', 'azerty', 'qwerty', 'tennis', 'tcsay', 'sayada', 'tunisie',
'bienvenue', 'welcome',
  '123456', '1234567890', 'admin', 'soleil', 'football',
];

/**
 * Règles d'un nouveau mot de passe : 10 caractères au moins, au moins une lettre et un chiffre,
pas un mot courant,
 * ni le prénom, le nom ou l'identifiant de la personne. Renvoie le message à afficher, ou null
si le mot de passe convient.
 */
export function passwordProblem(password: string, person: { email?: string | null; cin?: string |
null; firstName?: string; lastName?: string }): string | null {
  if (password.length < 10) return 'Le mot de passe doit contenir au moins 10 caractères.';
  if (!/[A-Za-zÀ-ÿ]/.test(password) || !/\d/.test(password)) return 'Le mot de passe doit
contenir au moins une lettre et un chiffre.';
  const plain = password.toLowerCase().normalize('NFD').replace(/[\u0300-\u036f]/g, '');
  const personal = [person.email?.split('@')[0], person.cin, person.firstName, person.lastName]
    .map((x) => (x ?? '').toLowerCase().normalize('NFD').replace(/[\u0300-\u036f]/g, '').trim())
    .filter((x) => x.length >= 3);
  if (personal.some((x) => plain.includes(x))) return 'Le mot de passe ne doit pas contenir votre
nom, prénom, email ou CIN.';
  const letters = plain.replace(/[^a-z]/g, '');
  if (COMMON.some((w) => letters === w || plain.startsWith(w))) return 'Mot de passe trop courant
: choisissez-en un autre.';
  if (/^(.)\1+$/.test(plain) || /^(0123456789|1234567890|abcdefghij)/.test(plain)) return 'Mot de
passe trop simple : choisissez-en un autre.';
  return null;
}
```

Annexe B — Extrait : garde des autorisations

```
  async canActivate(context: ExecutionContext): Promise<boolean> {
    const targets = [context.getHandler(), context.getClass()];
    if (this.reflector.getAllAndOverride<boolean>(IS_PUBLIC, targets)) return true;

    const req = context.switchToHttp().getRequest<{ headers: Record<string, string | undefined>;
user?: AuthUser }>();
    const header = req.headers.authorization ?? '';
    const token = header.startsWith('Bearer ') ? header.slice(7) : '';
    const payload = token ? verifyToken(token) : null;
    if (!payload) throw new UnauthorizedException('Session expirée : reconnectez-vous.');

    const user = await this.prisma.user.findUnique({
      where: { id: payload.sub },
      include: { player: { select: { id: true } }, coach: { select: { id: true } } },
    });
    if (!user || !user.isActive) throw new UnauthorizedException('Compte désactivé ou
introuvable.');
    // Mot de passe changé (par l'utilisateur ou nouveaux identifiants envoyés par le club) :
anciennes sessions refusées.
    if (payload.pw !== passwordStamp(user.passwordHash)) throw new UnauthorizedException('Session
expirée : reconnectez-vous.');

    if (user.mustChangePassword && !this.reflector.getAllAndOverride<boolean>
(ALLOW_PENDING_PASSWORD, targets)) {
      throw new ForbiddenException('Changez votre mot de passe temporaire pour continuer.');
    }

    const spaces = spacesOf(user.roles);
    const wanted = req.headers['x-espace'] as Space | undefined;
    const space: Space | undefined = wanted && spaces.includes(wanted) ? wanted : spaces[0];
    if (!space) throw new ForbiddenException('Aucun rôle attribué à ce compte : contactez le
club.');
    const role = SPACE_ROLE[space];
    const permissions = space === 'admin' ? await this.permissionsService.forRoles(user.roles) :
[];

    const roles = this.reflector.getAllAndOverride<Role[] | undefined>(ROLES, targets);
    if (roles?.length && !roles.includes(role)) {
      throw new ForbiddenException('Accès refusé pour votre rôle.');
    }
    const perms = this.reflector.getAllAndOverride<string[] | undefined>(PERMS, targets);
    if (perms?.length && (space !== 'admin' || !perms.some((p) => permissions.includes(p)))) {
      throw new ForbiddenException('Accès refusé : autorisation manquante (voir le président du
club).');
    }

  …
}
```

Annexe C — Commandes d'exploitation

Commande	Rôle
`npm run start:dev`	Démarre l'API en développement
`npm run dev` (web)	Démarre l'interface web
`npm run db:status`	Affiche les changements de structure en attente
`npm run db:migrate`	Sauvegarde puis applique les changements (après confirmation)
`npm run db:backup -- --keep 30`	Sauvegarde complète, garde les 30 dernières
`npm run db:restore`	Restaure une sauvegarde dans une base neuve
`npm run db:init-prod`	Prépare une base de production vierge (une seule fois)

Tableau 9 — Commandes d'exploitation